

Automated Analysis of Diffie-Hellman Protocols and Advanced Security Properties

Benedikt Schmidt, Simon Meier, Cas Cremers, David Basin
Institute of Information Security, ETH Zurich, Switzerland

Abstract—We present a general approach for the symbolic analysis of security protocols that use Diffie-Hellman exponentiation to achieve advanced security properties. We model protocols as multiset rewriting systems and security properties as first-order formulas. We analyze them using a novel constraint-solving algorithm that supports both falsification and verification, even in the presence of an unbounded number of protocol sessions. The algorithm exploits the finite variant property and builds on ideas from strand spaces and proof normal forms. We demonstrate the scope and the effectiveness of our algorithm on non-trivial case studies. For example, the algorithm successfully verifies the NAXOS protocol with respect to a symbolic version of the eCK security model.

I. INTRODUCTION

Authenticated Key Exchange (AKE) protocols are widely used components in modern network infrastructures. They assume a Public-Key Infrastructure and use the public keys to establish shared session keys over an untrusted channel. Recent AKE protocols use Diffie-Hellman (DH) exponentiation to achieve advanced security properties, namely secrecy and authentication properties in the presence of adversaries who are significantly more powerful than the classical Dolev-Yao adversary. For example, in the eCK model [1], the adversary may corrupt random number generators and dynamically compromise long-term keys and session keys.

As witnessed by the numerous attacks on published protocols, e.g. [2]–[5], designing AKE protocols is error-prone. It is therefore desirable to formally verify them before deployment, ideally automatically and with respect to an unbounded number of sessions. In this paper, we use a symbolic model of DH exponentiation to enable automatic verification. Our model supports DH exponentiation and an abelian group of exponents. This allows the adversary to cancel out DH exponents using exponentiation with their inverse. Similar to previous work on automatic symbolic analysis [6]–[8], we do not model multiplication in the DH group and addition of exponents.

There are no existing approaches capable of automatically verifying recent AKE protocols in models combining advanced security properties, unbounded sessions, and DH exponentiation. Existing approaches either bound the number of sessions [7], [8], fail to model the required adversary capabilities [6], [9]–[11], do not consider inverses in the group of DH exponents [12]–[14], or faithfully model the adversary, but do not support DH exponentiation [15], [16]. In this paper, we give a general approach to security protocol

verification, which is capable of automatically verifying AKE protocols in models as described above.

Contributions: First, we give an expressive and general security protocol model, which uses multiset rewriting to specify protocols and adversary capabilities, a guarded fragment [17] of first-order logic to specify security properties, and equational theories to model the algebraic properties of cryptographic operators.

Second, we give a novel constraint-solving algorithm for the falsification and verification of security protocols specified in our model for an unbounded number of sessions. We give a full proof of its correctness along with proofs of all theorems and assertions in this paper in the extended version [18].

Third, we implemented our algorithm in a tool, the TAMARIN prover [19], and validated its effectiveness on a number of non-trivial case studies. Despite the undecidability of the verification problem, our algorithm performs well: it terminates in the vast majority of cases, and the times for falsification and verification are in the range of a few seconds. This makes TAMARIN well-suited for the automated analysis of security protocols that use DH exponentiation to achieve advanced security properties.

Organization: We introduce notation in Section II and provide background on the security properties of AKE protocols in Section III. In Section IV, we define our protocol model. We present the theory underlying our constraint-solving algorithm in Section V and the algorithm in Section VI. We perform case studies in Section VII, compare with related work in Section VIII, and conclude in Section IX.

II. NOTATIONAL PRELIMINARIES

S^* denotes the set of sequences over S . For a sequence s , we write s_i for the i -th element, $|s|$ for the length of s , and $\text{idx}(s) = \{1, \dots, |s|\}$ for the set of indices of s . We write $\vec{s}$ to emphasize that s is a sequence. We use $[]$ to denote the empty sequence, $[s_1, \dots, s_k]$ to denote the sequence s where $|s| = k$, and $s \cdot s'$ to denote the concatenation of the sequences s and s' . $S^\#$ denotes the set of finite multisets with elements from S . We also use the superscript $\#$ to denote the usual operations on multisets such as $\cup^\#$. For a sequence s , $\text{mset}(s)$ denotes the corresponding multiset and $\text{set}(s)$ the corresponding set. We also use $\text{set}(m)$ for multisets m .

We write $\text{vars}(t)$ for the set of all variables in t , and $\text{fvars}(F)$ for the set of all variables that have free occurrences in a formula F . For a function f , we write $f[a \mapsto b]$ to denote the function that maps a to b and c to $f(c)$, for all $c \neq a$.

Figure 1. The NAXOS protocol.

$$\begin{array}{ll}
(1) \text{ dec}(\text{enc}(m, k), k) \simeq m & (6) x * 1 \simeq x \\
(2) \text{ fst}(\langle x, y \rangle) \simeq x & (7) x * x^{-1} \simeq 1 \\
(3) \text{ snd}(\langle x, y \rangle) \simeq y & (8) (x^{-1})^{-1} \simeq x \\
(4) x * (y * z) \simeq (x * y) * z & (9) (x \wedge y) \wedge z \simeq x \wedge (y * z) \\
(5) x * y \simeq y * x & (10) x \wedge 1 \simeq x
\end{array}$$

Figure 2. Equations that constitute E_{DH} .

The equational theory E_{DH} generated by the equations in Figure 2 formalizes the semantics of the function symbols in Σ_{DH} . It consists of equations for decryption and projection (1–3), exponentiation (9–10), and the theory of abelian groups for the exponents (4–8). Equation (9) states that repeated exponentiation in a DH group corresponds to multiplication of the exponents.

As an example, consider the term $((g \wedge a) \wedge b) \wedge a^{-1}$, which results from exponentiating g with a , followed by b , followed by a inverse. This is equal to $g \wedge ((a * b) * a^{-1})$ because of (9) and can be further simplified to $g \wedge b$ using (4–7).

Note that our approach supports the combination of Equations (2–10) modeling DH exponentiation and pairing with an arbitrary subterm-convergent rewriting theory for the user-defined cryptographic operators (see [18]). A rewriting theory R is subterm-convergent if it is convergent and for each rule $l \rightarrow r \in R$, r is either a proper subterm of l or is ground and in normal form with respect to R . One can therefore extend Σ_{DH} and E_{DH} with asymmetric encryption, signatures, and similar operators.

Note that our equational theory does not support protocols that perform multiplication in the DH group G . To define such protocols, an additional function symbol $\times$ denoting multiplication in G is required. The function symbol $*$ denotes multiplication in the group of exponents, which is a different operation. For example, the equality $(g \wedge a \times g \wedge b) \wedge c = (g \wedge a) \wedge c \times (g \wedge b) \wedge c$ holds in all DH groups, but does usually not hold if we replace $\times$ by $*$. Moreover, addition of exponents must be modeled for such protocols to avoid missing attacks. Consider the example protocol that randomly choses two exponents a and b , sends these exponents, receives some exponent x , and checks if $g \wedge a \times g \wedge b = g \wedge x$. This check succeeds if and only if $x = a + b$.

1) Transition System State: We model the states of our transition system as finite multisets of facts. We use a fixed set of fact symbols to encode the adversary’s knowledge, freshness information, and the messages on the network. The remaining fact symbols are used to represent the protocol state. Formally, we assume an unsorted signature Σ_{Fact} partitioned into *linear* and *persistent* fact symbols. We define the set of *facts* as the set $\mathcal{F}$ consisting of all facts $F(t_1, \dots, t_k)$ such that $t_i \in \mathcal{T}$ and $F \in \Sigma_{Fact}^k$. We denote the set of *ground facts* by $\mathcal{G}$. We say that a fact $F(t_1, \dots, t_k)$ is *linear* if F is linear and *persistent* if F is persistent.

Linear facts model resources that can only be consumed

once, whereas persistent facts model inexhaustible resources that can be consumed arbitrarily often. In the rest of the paper, we assume that Σ_{Fact} consists of an arbitrary number of protocol-specific fact symbols to describe the protocol state and the following special fact symbols. A persistent fact $K(m)$ denotes that m is known to the adversary. A linear fact $\text{Out}(m)$ denotes that the protocol has sent the message m , which can be received by the adversary. A linear fact $\text{In}(m)$ denotes that the adversary has sent the message m , which can be received by the protocol. A linear fact $\text{Fr}(n)$ denotes that the fresh name n was freshly generated.

2) Adversary, Protocol, and Freshness Rules: To specify the possible transitions by the adversary and the honest participants, we use *labeled multiset rewriting*. A *labeled multiset rewriting rule* is a triple (l, a, r) with $l, a, r \in \mathcal{F}^*$, denoted $l \rightarrow a \rightarrow r$. We often suppress the brackets around the sequences l , a , and r when writing rules. For $ri = l \rightarrow a \rightarrow r$, we define the *premises* as $\text{prems}(ri) = l$, the *actions* as $\text{acts}(ri) = a$, and the *conclusions* as $\text{concs}(ri) = r$. We use $\text{ginsts}(R)$ to denote the *set of ground instances* of a set of labeled multiset rewriting rules R .

There are three types of rules. A rule for fresh name generation, the message deduction rules, and the rules specifying the protocol and the adversary’s capabilities. All fresh names are created with the rule $\text{FRESH} = ([\] \rightarrow \text{Fr}(x:\text{fresh}))$. This is the only rule that produces Fr facts and we consider only runs with unique instances of this rule, i.e., the same fresh name is never generated twice.

We use the following set of *message deduction rules*.

$$\begin{aligned}
MD = \{ & \text{Out}(x) \rightarrow [\] \rightarrow K(x), \quad K(x) \rightarrow [\ K(x) \] \rightarrow \text{In}(x) \\
& \cup \{ [\] \rightarrow K(x:\text{pub}), \quad \text{Fr}(x:\text{fresh}) \rightarrow [\] \rightarrow K(x:\text{fresh}) \\
& \cup \{ K(x_1), \dots, K(x_k) \rightarrow [\] \rightarrow K(f(x_1, \dots, x_k)) \mid f \in \Sigma_{DH}^k \}
\end{aligned}$$

The rules in the first line allow the adversary to receive messages from the protocol and send messages to the protocol. The $K(x)$ action in the second rule makes the messages sent by the adversary observable in a protocol’s trace. We exploit this to specify secrecy properties. The rules in the second line allow the adversary to learn public names and freshly generated names. The remaining rules allow the adversary to apply functions from Σ_{DH} to known messages.

A *protocol rule* is a multiset rewriting rule $l \rightarrow a \rightarrow r$ such that **(P1)** it does not contain fresh names, **(P2)** K and Out facts do not occur in l , **(P3)** K , In , and Fr facts do not occur in r , and **(P4)** $\text{vars}(r) \subseteq \text{vars}(l) \cup \mathcal{V}_{\text{pub}}$. A *protocol* is a finite set of protocol rules. Note that our formal notion of a protocol encompasses both the rules executed by the honest participants and the adversary’s capabilities, like revealing long-term keys. Condition **P1** and the restriction on the usage of Fr facts from **P3**, which also hold for the message deduction rules, ensure that all fresh names originate from instances of the FRESH rule.

3) *Transition Relation*: The labeled transition relation $\rightarrow_P \subseteq \mathcal{G}^\# \times \mathcal{P}(\mathcal{G}) \times \mathcal{G}^\#$ for a protocol P is defined by the transition rule

$$\frac{l \vdash a \rightarrow_{E_{DH}} \text{ginsts}(P \cup MD \cup \{\text{FRESH}\}) \quad lfacts(l) \subseteq^\# S \quad pfacts(l) \subseteq \text{set}(S)}{S \xrightarrow{\text{set}(a)}_P ((S \setminus^\# lfacts(l)) \cup^\# mset(r))},$$

where $lfacts(l)$ is the multiset of all linear facts in l and $pfacts(l)$ is the set of all persistent facts in l . This transition rule models rewriting the state with a ground instance of a protocol rule, a message deduction rule, or the FRESH rule. Since we perform multiset rewriting modulo E_{DH} , we use $\in_{E_{DH}}$ for the rule instance. As linear facts are consumed upon rewriting, we use multiset inclusion to check that all facts in $lfacts(l)$ occur sufficiently many times in S . For persistent facts, we only check that every fact in $pfacts(l)$ occurs in S . To obtain the successor state, we remove the consumed linear facts and add the generated facts. The action associated to the transition is the set of actions of the rule instance.

A *trace* is a sequence of sets of ground facts denoting the sequence of actions that happened during a protocol's execution. We model the executions of a security protocol P by its set of traces defined as

$$\begin{aligned} \text{traces}(P) = \{ & [A_1, \dots, A_n] \\ & | \exists S_1, \dots, S_n \in \mathcal{G}^\#. \emptyset^\# \xrightarrow{A_1}_P \dots \xrightarrow{A_n}_P S_n \\ & \wedge \forall i \neq j. \forall x. (S_{i+1} \setminus^\# S_i) = \{\text{Fr}(x)\}^\# \Rightarrow \\ & (S_{j+1} \setminus^\# S_j) \neq \{\text{Fr}(x)\}^\# \} \end{aligned}$$

The second conjunct ensures that each instance of the FRESH rule is used at most once in a trace. Each consumer of a $\text{Fr}(n)$ fact therefore obtains a different fresh name. Transitions labeled with $\emptyset$ are silent. We therefore define the *observable trace* $\bar{tr}$ of a trace tr as the subsequence of all non-silent actions in tr .

B. Security Properties

We use two-sorted first-order-logic to specify security properties. This logic supports quantification over both messages and timepoints. We thus introduce the sort *temp* for timepoints and write $\mathcal{V}_{temp}$ for the set of *temporal variables*.

A *trace atom* is either *false* $\perp$, a *term equality* $t_1 \approx t_2$, a *timepoint ordering* $i < j$, a *timepoint equality* $i \doteq j$, or an *action* $f@i$ for a fact f and a timepoint i . A *trace formula* is a first-order formula over trace atoms.

To define the semantics of trace formulas, we associate a domain $\mathbf{D}_s$ with each sort s . The domain for temporal variables is $\mathbf{D}_{temp} = \mathbb{Q}$ and the domains for messages are $\mathbf{D}_{msg} = \mathcal{M}$, $\mathbf{D}_{fresh} = FN$, and $\mathbf{D}_{pub} = PN$. We say a function θ from $\mathcal{V}$ to $\mathbb{Q} \cup \mathcal{M}$ is a *valuation* if it respects sorts, i.e., $\theta(\mathcal{V}_s) \subseteq \mathbf{D}_s$ for all sorts s . For a term t , we write $t\theta$ for the application of the homomorphic extension of θ to t .

For an equational theory E , the *satisfaction relation* $(tr, \theta) \models_E \varphi$ between traces tr , valuations θ , and trace formulas φ is defined as follows.

$$\begin{aligned} (tr, \theta) \models_E f@i & \quad \text{iff } \theta(i) \in \text{idx}(tr) \text{ and } f\theta \in_{E_{DH}} tr_{\theta(i)} \\ (tr, \theta) \models_E i < j & \quad \text{iff } \theta(i) < \theta(j) \\ (tr, \theta) \models_E i \doteq j & \quad \text{iff } \theta(i) = \theta(j) \\ (tr, \theta) \models_E t_1 \approx t_2 & \quad \text{iff } t_1\theta =_E t_2\theta \\ (tr, \theta) \models_E \neg\varphi & \quad \text{iff not } (tr, \theta) \models_E \varphi \\ (tr, \theta) \models_E \varphi \wedge \psi & \quad \text{iff } (tr, \theta) \models_E \varphi \text{ and } (tr, \theta) \models_E \psi \\ (tr, \theta) \models_E \exists x:s. \varphi & \quad \text{iff there is } u \in \mathbf{D}_s \text{ such that} \\ & \quad (tr, \theta[x \mapsto u]) \models_E \varphi \end{aligned}$$

The semantics of the remaining logical connectives and quantifiers are defined by translation to the given fragment as usual. Overloading notation, we write $tr \models_E \varphi$ if $(tr, \theta) \models_E \varphi$ for all θ . For a set of traces TR , we write $TR \models_E \varphi$ if $tr \models_E \varphi$ for all $tr \in TR$. We say that a *protocol* P *satisfies* φ , written $P \models_{E_{DH}} \varphi$, if $\text{traces}(P) \models_{E_{DH}} \varphi$.

C. Example: Security of NAXOS in the eCK Model

We formalize the NAXOS protocol for the eCK model using the rules in Figure 3. We include two free function symbols h_1 and h_2 in Σ_{DH} . The first rule models the generation and registration of long-term asymmetric keys. An exponent lk_A is randomly chosen and stored as the long-term key of an agent A . The persistent facts $!Ltk(A, lk_A)$ and $!Pk(A, g^{lk_A})$ denote the association between A and his long-term private and public keys. The public key is additionally sent to the adversary.

In the rules modeling the initiator and responder, each protocol thread chooses a unique ephemeral key esk_x , which we also use to identify the thread. The first initiator rule chooses the actor I and the intended partner R , looks up I 's long-term key, and sends the half-key hk_I . The fact $\text{Init}_1(esk_I, I, R, lk_I, hk_I)$ then stores the state of thread esk_I and the fact $!Ephk(esk_I, esk_I)$ is added to allow the adversary to reveal the ephemeral key esk_I (the second argument) of the thread esk_I (the first argument). The second initiator rule reads the thread's state, looks up the public key of the intended partner, and receives the half-key Y . The key k_I is then computed. The action $\text{Accept}(esk_I, I, R, k_I)$ denotes that the thread esk_I finished with the given parameters.

To specify when two threads are intended communication partners ("matching sessions"), we include $\text{Sid}(esk_I, sid)$ and $\text{Match}(esk_I, sid')$ actions. A thread s' matches a thread s if there exists a sid such that $\text{Sid}(s', sid)$ and $\text{Match}(s, sid)$ occur in the trace. By appropriately defining the Match actions and session identifier sid , various definitions of matching can be modeled [21].

Finally, the fact $!Sessk(esk_I, k_I)$ is added to the second initiator rule to allow revealing the session key k_I . The responder rule works analogously. The final three rules model

Generate long-term keypair:

$\text{Fr}(lk_A) \dashv \text{!Ltk}(A:\text{pub}, lk_A), \text{!Pk}(A, g \wedge lk_A), \text{Out}(g \wedge lk_A)$

Initiator step 1:

$\text{Fr}(esk_I), \text{!Ltk}(I, lk_I)$

$\dashv \text{!Init}_1(esk_I, I, R:\text{pub}, lk_I, hk_I), \text{!Ephk}(esk_I, esk_I), \text{Out}(hk_I)$

where $hk_I = g \wedge h_1(esk_I, lk_I)$

Initiator step 2:

$\text{Init}_1(esk_I, I, R, lk_I, hk_I), \text{!Pk}(R, pk_R), \text{In}(Y)$

$\dashv \text{!Accept}(esk_I, I, R, k_I), \text{Sid}(esk_I, \langle \text{Init}, I, R, hk_I, Y \rangle)$
 $, \text{Match}(esk_I, \langle \text{Resp}, R, I, hk_I, Y \rangle) \dashv \text{!Sessk}(esk_I, k_I)$

where $k_I = h_2(Y \wedge lk_I, pk_R \wedge h_1(esk_I, lk_I), Y \wedge h_1(esk_I, lk_I), I, R)$

Responder:

$\text{Fr}(esk_R), \text{!Ltk}(R, lk_R), \text{!Pk}(I, pk_I), \text{In}(X)$

$\dashv \text{!Accept}(esk_R, R, I, k_R), \text{Sid}(esk_R, \langle \text{Resp}, R, I, X, hk_R \rangle)$
 $, \text{Match}(esk_R, \langle \text{Init}, I, R, X, hk_R \rangle) \dashv$

$\text{!Sessk}(esk_R, k_R), \text{!Ephk}(esk_R, esk_R), \text{Out}(g \wedge h_1(esk_R, lk_R))$

where $hk_R = g \wedge h_1(esk_R, lk_R)$, and

$k_R = h_2(pk_I \wedge h_1(esk_R, lk_R), X \wedge lk_R, X \wedge h_1(esk_R, lk_R), I, R)$

Key Reveals for the eCK model:

$\text{!Sessk}(tid, k) \dashv \text{!SesskRev}(tid) \dashv \text{Out}(k)$
 $\text{!Ltk}(A, lk_A) \dashv \text{!LtkRev}(A) \dashv \text{Out}(lk_A)$
 $\text{!Ephk}(tid, esk_A) \dashv \text{!EphkRev}(tid) \dashv \text{Out}(esk_A)$

Figure 3. Multiset rewriting rules formalizing NAXOS.

that, in the eCK model, the adversary can reveal any session, long-term, or ephemeral key. We model the restrictions on key reveals as part of the security property and thus record all key reveals in the trace.

We formalize security in the eCK model by the formula in Figure 4, which is a one-to-one mapping of the original definition of eCK security given in [1]. Intuitively, the formula states that if the adversary knows the session key of a thread esk_I , then he must have performed forbidden key reveals. The left-hand side of the implication states that the key k is known and the right-hand side disjunction states the restrictions on key reveals. We describe each disjunct in the comment above it. Further motivation and variants of these restrictions can be found in [1], [21]. Note that the eCK model formalizes weak Perfect Forward Secrecy (weak PFS), as it only allows for a long-term key reveal of the intended partner if there is a matching session. To obtain a variant of eCK formalizing PFS, we can replace the last line with

$$\vee (\exists i_5. \text{LtkRev}(B)@i_5 \wedge i_5 \leq i_1)) \Big) \Big).$$

This allows the adversary to reveal the long-term key of the intended partner after the test thread is finished or if there is a matching session. The NAXOS protocol does not satisfy this property, as reported in Table I in Section VII.

Note that some protocols require modeling inequality conditions, e.g., the TS1-2004 [22] protocol, which assumes that an agent never executes a session with himself. We

$$\forall i_1 i_2 s A B k. (\text{Accept}(s, A, B, k)@i_1 \wedge K(k)@i_2) \Rightarrow$$

// If the session key of the test thread s is known, then
 // s must be "not clean". Hence either there is a
 // session key reveal for s ,

$$((\exists i_3. \text{SesskRev}(s)@i_3)$$

// or a session key reveal for a matching session,

$$\vee (\exists s' i_3 i_4 \text{sid}. (\text{Sid}(s', \text{sid})@i_3 \wedge \text{Match}(s, \text{sid})@i_4) \\ \wedge (\exists i_5. \text{SesskRev}(s')@i_5))$$

// or if a matching session exists,

$$\vee (\exists s' i_3 i_4 \text{sid}. (\text{Sid}(s', \text{sid})@i_3 \wedge \text{Match}(s, \text{sid})@i_4)$$

// both lk_A and esk_A , or both lk_B and esk_B are revealed,

$$\wedge ((\exists i_5 i_6. \text{LtkRev}(A)@i_5 \wedge \text{EphkRev}(s)@i_6)$$

$$\vee (\exists i_5 i_6. \text{LtkRev}(B)@i_5 \wedge \text{EphkRev}(s')@i_6)))$$

// or if no matching session exists,

$$\vee (\neg (\exists s' i_3 i_4 \text{sid}. (\text{Sid}(s', \text{sid})@i_3 \wedge \text{Match}(s, \text{sid})@i_4))$$

// either both lk_A and esk_A , or lk_B are revealed.

$$\wedge ((\exists i_5 i_6. \text{LtkRev}(A)@i_5 \wedge \text{EphkRev}(s)@i_6)$$

$$\vee (\exists i_5. \text{LtkRev}(B)@i_5))) \Big) \Big)$$

Figure 4. eCK security definition.

model inequality conditions in two steps. First, we include $\text{Neq}(s, t)$ facts in the actions of rules that require the terms s and t to be unequal. Second, we replace the considered security property φ with $(\neg(\exists i x. \text{Neq}(x, x)@i)) \Rightarrow \varphi$ to restrict the analysis to traces where all inequality conditions hold. This filtering construction also works for enforcing other restrictions on traces, e.g., the uniqueness of certain actions.

V. NORMAL DEPENDENCY GRAPHS

For symbolic attack-search algorithms, there are several drawbacks to the multiset rewriting semantics given in the previous section. First, incrementally constructing attacks is difficult with (action-)traces, as they contain neither the history of past states nor the causal dependencies between steps. Second, symbolic reasoning modulo E_{DH} is difficult because E_{DH} contains cancellation equations. For example, if the adversary knows $t = na * x$ for a nonce na , we cannot conclude that na has been used in the construction of t , as x could be equal to na^{-1} . Third, the message deduction rules allow for redundant steps such as first encrypting a cleartext and then decrypting the resulting ciphertext. For search algorithms, it is useful to impose normal-form conditions on message deduction to avoid exploring such redundant steps.

We take the following approach. First, we define dependency graphs. They consist of the sequence of rewriting rule instances corresponding to a protocol execution and their causal dependencies, similar to strand spaces [23]. Afterwards, we show that we can use dependency graphs modulo AC , an equational theory without cancellation equations, instead of dependency graphs modulo E_{DH} .

Finally, we define normal message deductions and the corresponding normal dependency graphs. We also show that normal dependency graphs are weakly trace equivalent to the multiset rewriting semantics.

A. Dependency Graphs

We use dependency graphs to represent protocol executions together with their causal dependencies. A dependency graph consists of nodes labeled with rule instances and dependencies between the nodes. We first present an example of a dependency graph and then give its formal definition.

Example 1 (Dependency Graph). Consider the protocol

$$P = \{ [\text{Fr}(x), \text{Fr}(k)] \rightarrow [\text{St}(x, k), \text{Out}(\text{enc}(x, k)), \text{Key}(k)] \\ , [\text{St}(x, k), \text{In}(\langle x, x \rangle)] \rightarrow [\text{Fin}(x, k)] \rightarrow [] \\ , [\text{Key}(k)] \rightarrow [\text{Rev}(k)] \rightarrow [\text{Out}(k)] \} .$$

Figure 5 shows a dependency graph for an execution of P . We use inference rule notation with the actions on the right for rule instances. Nodes 1 and 2 are rule instances that create fresh names. Node 3 is an instance of the first protocol rule. Node 4 is an instance of the key reveal rule. Nodes 5–9 are instances of message deduction rules and denote that the adversary receives a ciphertext and its key, decrypts the ciphertext, pairs the resulting cleartext with itself, and sends the result to an instance of the second protocol rule, Node 10. The edges denote causal dependencies: an edge from a conclusion of node i to a premise of node j denotes that the corresponding fact is generated by i and consumed by j . Since this is a dependency graph modulo E_{DH} , it is sufficient that each pair of generated and consumed facts is equal modulo E_{DH} .

Formally, let E be an equational theory and R be a set of multiset rewriting rules. We say that $dg = (I, D)$ is a *dependency graph modulo E for R* if $I \in (\text{ginsts}(R \cup \{\text{FRESH}\}))^*$, $D \subseteq \mathbb{N}^2 \times \mathbb{N}^2$, and dg satisfies the conditions **DG1–4** listed below. To state these conditions, we introduce the following definitions. We call $\text{idx}(I)$ the *nodes* and D the *edges* of dg . We write $(i, u) \rightarrow (j, v)$ for the edge $((i, u), (j, v))$. Let $I = [l_1 \rightarrow a_1, \dots, l_n \rightarrow a_n]$. The *trace* of dg is $\text{trace}(dg) = [\text{set}(a_1), \dots, \text{set}(a_n)]$. A *conclusion* of dg is a pair (i, u) such that i is a node of dg and $u \in \text{idx}(r_i)$. The corresponding *conclusion fact* is $(r_i)_u$. A *premise* of dg is a pair (i, u) such that i is a node of dg and $u \in \text{idx}(l_i)$. The corresponding *premise fact* is $(l_i)_u$. A conclusion or premise is *linear* if its fact is linear.

- DG1** For every edge $(i, u) \rightarrow (j, v) \in D$, it holds that $i < j$ and the conclusion fact of (i, u) is equal modulo E to the premise fact of (j, v) .
- DG2** Every premise of dg has exactly one incoming edge.
- DG3** Every linear conclusion of dg has at most one outgoing edge.
- DG4** The FRESH rule instances in I are unique.

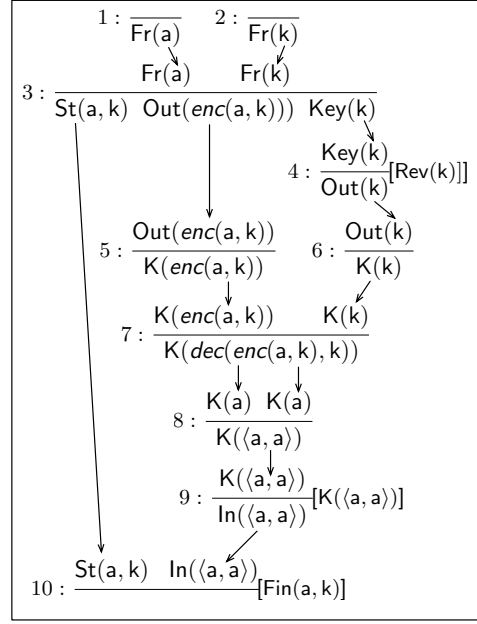


Figure 5. Dependency graph modulo E_{DH} .

We denote the set of all dependency graphs modulo E for R by $\text{dgraphs}_E(R)$.

Note that, for all protocols P , the multiset rewriting semantics given in Section IV and the dependency graphs modulo E_{DH} for $P \cup MD$ have the same set of traces, i.e., $\text{traces}(P) =_{E_{DH}} \{\text{trace}(dg) \mid dg \in \text{dgraphs}_{E_{DH}}(P \cup MD)\}$.

B. Dependency Graphs modulo AC

We now switch to a semantics based on dependency graphs modulo AC . We use standard notions from order-sorted rewriting [24] and proceed in two steps.

First, we define AC as the equational theory generated by Equations (4–5) from Figure 2 and DH as the rewriting system obtained by orienting Equations (1–3, 9–10) from Figure 2 and all equations from Figure 6 from left to right. $DH \uplus AC$ is an equational presentation of E_{DH} and DH is AC -convergent and AC -coherent. We can therefore define $t \downarrow_{DH}$ as the normal form of t with respect to DH, AC -rewriting and have $t =_{E_{DH}} s$ iff $t \downarrow_{DH} =_{AC} s \downarrow_{DH}$. We say that t is $\downarrow_{DH}$ -normal if $t =_{AC} t \downarrow_{DH}$. We say a dependency graph $dg = (I, D)$ is $\downarrow_{DH}$ -normal if all rule instances in I are $\downarrow_{DH}$ -normal.

Second, E_{DH} has the finite variant property [25] for this presentation, which allows us to perform symbolic reasoning about normalization. More precisely, for all terms t , there is a finite set of substitutions $\{\tau_1, \dots, \tau_k\}$ such that for all substitutions σ , there is an $i \in \{1, \dots, k\}$ and a substitution σ' with $(t\sigma) \downarrow_{DH} =_{AC} ((\tau_i) \downarrow_{DH}) \sigma'$ and $(x\sigma) \downarrow_{DH} =_{AC} x\tau_i \sigma'$ for all $x \in \text{vars}(t)$. We call $\{(\tau_i \downarrow_{DH}, \tau_i) \mid 1 \leq i \leq k\}$ a *complete set of DH, AC -variants of t* . For a given term t , we use folding variant narrowing [24] to compute such a set,

$$\begin{array}{ll}
(1) (x^{-1} * y)^{-1} \simeq x * y^{-1} & (6) 1^{-1} \simeq 1 \\
(2) x^{-1} * y^{-1} \simeq (x * y)^{-1} & (7) x * 1 \simeq x \\
(3) x * (x * y)^{-1} \simeq y^{-1} & (8) (x^{-1})^{-1} \simeq x \\
(4) x^{-1} * (y^{-1} * z) \simeq (x * y)^{-1} * z & (9) x * (x^{-1} * y) \simeq y \\
(5) (x * y)^{-1} * (y * z) \simeq x^{-1} * z & (10) x * x^{-1} \simeq 1
\end{array}$$

Figure 6. Lankford's presentation of the abelian group axioms

which we denote by $[t]^{DH}$. Overloading notation, we also denote $\{s \mid (s, \tau) \in [t]^{DH}\}$ by $[t]^{DH}$.

It is straightforward to extend these notions to multiset rewriting rules by considering rules as terms and the required new function symbols as free. We can then show that $dgraphs_{EDH}(R) \downarrow_{DH} \subseteq_{AC} dgraphs_{AC}([R]^{DH})$ for all sets of multiset rewriting rules R . If we restrict the right hand side to $\downarrow_{DH}$ -normal dependency graphs, then the two sets are equal modulo AC .

Example 2. To normalize the graph $dg = (I, D)$ in Figure 5 with respect to $\downarrow_{DH}$, it suffices to replace I_7 with $ri = K(\text{enc}(a, k)), K(k) \rightarrow K(a)$, calling the result dg' . Since ri is not an instance of the decryption rule $rdec = K(x), K(y) \rightarrow K(\text{dec}(x, y))$ or any other rule in $P \cup MD$, dg' is not in $dgraphs_{AC}(P \cup MD)$. However, ri is an instance of $K(\text{enc}(x, y)), K(y) \rightarrow K(x)$, which is a DH, AC -variant of $rdec$ and therefore in $[MD]^{DH}$. Hence, $dg' \in dgraphs_{AC}([P \cup MD]^{DH})$.

C. Normal Dependency Graphs

We first define the class of $*$ -restricted protocols, which do not multiply exponents. We then define rules for normal message deduction and the corresponding normal dependency graphs, which are weakly trace equivalent to the multiset rewriting semantics for $*$ -restricted protocols.

1) **-restricted Protocols:* The following restriction ensures that protocols do not multiply exponents and do not introduce products by other means. A protocol P is **-restricted* if, for each of its rules $l \rightarrow r$, (a) l does not contain the function symbols $*$, $\wedge$, $^{-1}$, fst , snd , and dec , and (b) r does not contain the function symbol $*$.

In general, condition (a) prevents protocol rules from pattern matching on reducible function symbols. Condition (b) prevents protocols from directly using multiplication, although repeated exponentiation is still allowed. Note that these restrictions are similar to those of previous work such as [6], [7] and are not a restriction in practice. Protocols that use multiplication in the group of exponents can usually be specified by using repeated exponentiation. Moreover, protocols that use multiplication in the DH group, such as MQV [26], cannot be specified anyway since $*$ denotes multiplication in the group of exponents.

For $*$ -restricted protocols, products that occur in positions that can be extracted by the adversary can always be

constructed by the adversary himself from their components.

2) *Normal Message Deduction:* Message deduction steps in dependency graphs modulo AC use rules from $[MD]^{DH}$. These rules still allow redundant steps. We now eliminate some of them by tagging the rules to limit their applicability.

We first partition $[MD]^{DH}$ into five subsets: communication rules for sending and receiving messages, multiplication rules consisting of all DH, AC -variants of the rule for multiplication, construction rules that apply a function symbol to arguments, deconstruction rules that extract a subterm from an argument, and the remaining exponentiation rules, which are all DH, AC -variants of the rule for exponentiation and are neither construction nor deconstruction rules.

We use tags to forbid two types of redundancies. First, we forbid using a deconstruction rule to deconstruct the result of a construction rule. This is analogous to restrictions for normal natural deduction proofs, adapted to the setting of message deduction [27]. Second, we forbid repeated exponentiation, which can always be replaced by a single exponentiation with the product of all exponents. In particular, we use the *deconstruction tags* $\downarrow$ and $\uparrow$ and the *exponentiation tags* exp and noexp . We use $\downarrow$ to tag K -facts where deconstruction is allowed and $\uparrow$ where deconstruction is forbidden. We use exp to tag K -facts that can be used as the base of an exponentiation and noexp where this is forbidden.

We obtain the *normal message deduction rules ND* shown in Figure 9 as follows. First, we add the COERCE rule to switch from message deconstruction to message construction, preserving the exponentiation tag. Second, we replace the multiplication rules by l -ary construction rules for multiplication. For $*$ -restricted protocols, these rules are sufficient to reason about products. Third, we use exponentiation tags as follows. The construction rule for exponentiation, the deconstruction rules for exponentiation, and the exponentiation rules use exp for the first premise (the base), a variable for the second premise, and noexp for the conclusion. The remaining rules use variables for the premises and exp for the conclusion. Finally, we use deconstruction tags as follows. We use $\uparrow$ for the conclusion of construction rules and $\downarrow$ for the first premise of deconstruction and exponentiation rules. This ensures that a deconstruction or exponentiation rule can never use the conclusion of a construction rule as its first premise. For the remaining premises of rules, we use $\uparrow$. For the remaining conclusions of rules, we use $\downarrow$. Note that all conclusions of exponentiation rules, including the ones that we do not show, are of the form $K_{\text{noexp}}^1(t \wedge s)$ and can therefore only be used by COERCE.

Example 3. Figure 7 shows five message deduction sub-graphs. In (a), the adversary decrypts a message that he earlier encrypted himself. Instead of performing these deductions, the adversary can directly use the conclusion $K(a)$ that is used by the encryption. The deduction from (a) is not possible with the normal message deduction rules ND because the

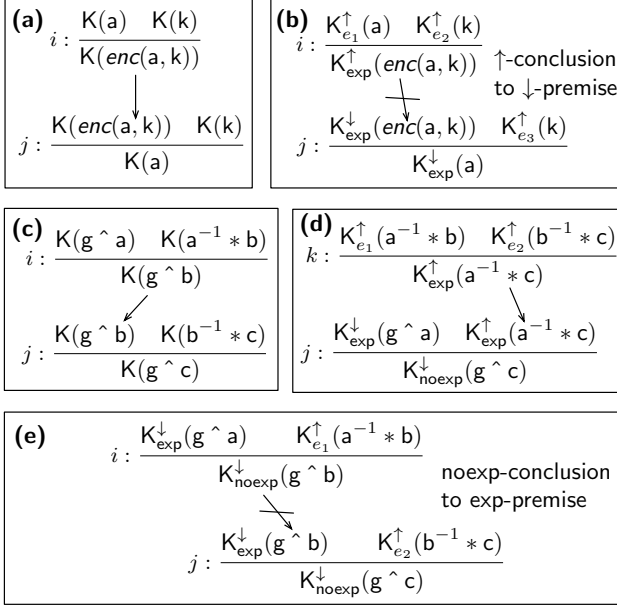


Figure 7. Message deduction subgraphs for encryption. We use $\nrightarrow$ for edges that are invalid because the source and target are not equal. We use $i, j, k \in \mathbb{N}$ and the exponentiation tags e_1, e_2 , and e_3 .

$\uparrow$ -tags and $\downarrow$ -tags prevent applying a deconstruction rule to the conclusion of a construction rule, as depicted in (b). In (c), the adversary performs a redundant step that involves repeated exponentiation. Note that an unbounded number of steps that add a new exponent and remove the previously added exponent can be inserted inbetween the two rules. This deduction is impossible with the normal message deduction rules because a conclusion with a **noexp**-tag cannot be used with a premise that requires an **exp**-tag, as depicted in (e). We can replace the repeated exponentiation with one multiplication and one exponentiation as depicted in (d).

3) Normal Dependency Graphs: We now define normal dependency graphs. They use the normal message deduction rules and enforce further normal-form conditions. To state the conditions, we define the *input components* of a term t as $\text{inp}(t)$, such that $\text{inp}(t^{-1}) = \text{inp}(t)$, $\text{inp}(\{t_1, t_2\}) = \text{inp}(t_1) \cup \text{inp}(t_2)$, $\text{inp}(t_1 * t_2) = \text{inp}(t_1) \cup \text{inp}(t_2)$, and $\text{inp}(t) = \{t\}$ otherwise. Intuitively, $\text{inp}(t)$ consists of the maximal subterms of t that are not products, pairs, or inverses.

Formally, a *normal dependency graph* for a protocol P is a dependency graph dg such that $dg \in \text{dgraphs}_{AC}([P]^{DH} \cup ND)$ and the following conditions are satisfied.

- N1** The dependency graph dg is $\downarrow_{DH}$ -normal.
- N2** No instance of COERCE deduces a pair or an inverse.
- N3** There is no multiplication rule that has a premise fact of the form $K_e^d(t * s)$.
- N4** All conclusion facts $K_e^d(t * s)$ are conclusions of a multiplication rule.
- N5** If there are two conclusions c and c' with conclusion facts $K_e^d(m)$ and $K_{e'}^d(m')$ such that $m =_{AC} m'$, then $c = c'$.

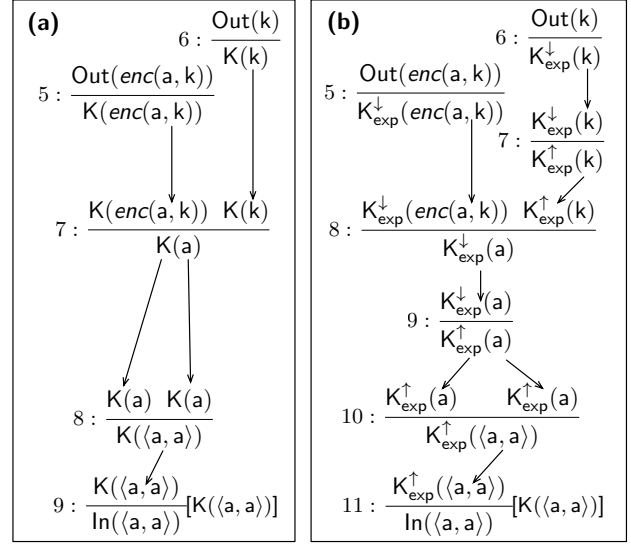


Figure 8. Examples of message deduction subgraphs of (a) a dependency graph modulo AC and (b) a normal dependency graph.

- N6** If there is a conclusion $(i, 1)$ with fact $K_e^d(m)$ and a conclusion $(j, 1)$ with fact $K_{e'}^d(m')$ such that $m =_{AC} m'$, then $i < j$ and j is an instance of COERCE or the construction rule for pairing or the one for inversion.
- N7** For all nodes $K_{\text{exp}}^d(s_1), K_e^d(t_1) \rightarrow K_{\text{noexp}}^d(s_2 \wedge t_2)$ such that s_2 is of sort *pub*, $\text{inp}(t_2) \not\subseteq \text{inp}(t_1)$.

We denote the set of all normal dependency graphs of P by $\text{ndgraphs}(P)$.

N1 ensures that all rule instances are $\downarrow_{DH}$ -normal. **N2** ensures that pairs and inverses are always completely deconstructed. **N3** and **N4** formalize that the adversary constructs all products directly by multiplying their components. This does not restrict the adversary since we limit ourselves to $*$ -restricted protocols. **N5** and **N6** ensure a restricted form of message uniqueness. **N7** forbids instances of exponentiation rules that can be replaced by instances of the construction rule for exponentiation where the base is a public name.

Note that normal dependency graphs allow exactly the same executions as our multiset rewriting semantics.

Lemma 1. For all $*$ -restricted protocols P ,

$$\overline{\text{traces}}(P) \downarrow_{DH} =_{AC} \{\overline{\text{trace}}(dg) \mid dg \in \text{ndgraphs}(P)\}.$$

Example 4. Figure 8 shows two message deduction subgraphs. Subfigure (a) shows the message deduction subgraph of a dependency graph modulo AC and (b) shows the message deduction subgraph of the corresponding normal dependency graph. We obtain (b) from (a) by renumbering the nodes and adding the required tags and COERCE nodes.

Example 5. Consider the exponentiation-construction node $i : K_{\text{exp}}^d(g \wedge a), K_{\text{exp}}^d(a^{-1} * b) \rightarrow K_{\text{noexp}}^d(g \wedge b)$. The conclusion of i either has no outgoing edge or a single edge to a COERCE node j . In the first case, the node i can be removed.

$$\begin{array}{l}
\text{Coerce rule:} \quad \text{COERCE} \frac{K_e^\downarrow(x)}{K_e^\uparrow(x)} \quad \text{Communication rules:} \quad \text{IRECV} \frac{\text{Out}(x)}{K_{\text{exp}}^\downarrow(x)} \quad \text{ISEND} \frac{K_e^\uparrow(x)}{\text{In}(x)} \llbracket K(x) \rrbracket \\
\\
\text{Construction rules:} \\
\frac{K_{\text{exp}}^\uparrow(x) \ K_e^\uparrow(y)}{K_{\text{noexp}}^\uparrow(x \wedge y)} \quad \frac{}{K_{\text{exp}}^\uparrow(x:\text{pub})} \quad \frac{\text{Fr}(x:\text{fresh})}{K_{\text{exp}}^\uparrow(x:\text{fresh})} \quad \frac{K_e^\uparrow(x)}{K_{\text{exp}}^\uparrow(x^{-1})} \quad \frac{}{K_{\text{exp}}^\uparrow(1)} \quad \frac{K_{e_1}^\uparrow(x) \ K_{e_2}^\uparrow(y)}{K_{\text{exp}}^\uparrow(\text{enc}(x, y))} \quad \frac{K_{e_1}^\uparrow(x) \ K_{e_2}^\uparrow(y)}{K_{\text{exp}}^\uparrow(\text{dec}(x, y))} \\
\\
\frac{K_e^\uparrow(x)}{K_{\text{exp}}^\uparrow(h(x))} \quad \frac{K_e^\uparrow(x)}{K_{\text{exp}}^\uparrow(\text{fst}(x))} \quad \frac{K_e^\uparrow(x)}{K_{\text{exp}}^\uparrow(\text{snd}(x))} \quad \frac{K_{e_1}^\uparrow(x) \ K_{e_2}^\uparrow(y)}{K_{\text{exp}}^\uparrow(\langle x, y \rangle)} \quad \frac{K_{e_1}^\uparrow(x_1) \ \dots \ K_{e_n}^\uparrow(x_n) \quad K_{e_{n+1}}^\uparrow(x_{n+1}) \ \dots \ K_{e_l}^\uparrow(x_l)}{K_{\text{exp}}^\uparrow((x_1 * \dots * x_n) * (x_{n+1} * \dots * x_l)^{-1})} \\
\\
\text{Deconstruction rules:} \\
\frac{K_{\text{exp}}^\downarrow(x \wedge y) \ K_e^\uparrow(y^{-1})}{K_{\text{noexp}}^\downarrow(x)} \quad \frac{K_{\text{exp}}^\downarrow(x \wedge y^{-1}) \ K_e^\uparrow(y)}{K_{\text{noexp}}^\downarrow(x)} \quad \frac{K_{\text{exp}}^\downarrow(x \wedge (y * z^{-1})) \ K_e^\uparrow(y^{-1} * z)}{K_{\text{noexp}}^\downarrow(x)} \\
\\
\frac{K_e^\downarrow(\langle x, y \rangle)}{K_{\text{exp}}^\downarrow(x)} \quad \frac{K_e^\downarrow(\langle x, y \rangle)}{K_{\text{exp}}^\downarrow(y)} \quad \frac{K_e^\downarrow(x^{-1})}{K_{\text{exp}}^\downarrow(x)} \quad \frac{K_{e_1}^\downarrow(\text{enc}(x, y)) \ K_{e_2}^\uparrow(y)}{K_{\text{exp}}^\downarrow(x)} \\
\\
\text{Exponentiation rules:} \quad \frac{K_{\text{exp}}^\downarrow(x \wedge y) \ K_e^\uparrow(z)}{K_{\text{noexp}}^\downarrow(x \wedge (y * z))} \quad \frac{K_{\text{exp}}^\downarrow(x \wedge y) \ K_e^\uparrow(y^{-1} * z)}{K_{\text{noexp}}^\downarrow(x \wedge z)} \quad \dots \quad \frac{K_{\text{exp}}^\downarrow(x \wedge (y * z^{-1})) \ K_e^\uparrow(a * b^{-1})}{K_{\text{noexp}}^\downarrow(x \wedge (y * a * (z * b)^{-1}))}
\end{array}$$

Figure 9. Normal message deduction rules ND . Rules containing variables e or e_i denote all variants where these are replaced by noexp or exp . Rules containing n and l denote all variants for $n \geq 1$ and $l \geq 2$. There are 42 exponentiation rules computed from the DH, AC -variants of the exponentiation rule.

In the second case, the nodes i and j can be replaced by $j : K_{\text{exp}}^\uparrow(\mathbf{g}), K_{\text{exp}}^\uparrow(\mathbf{b}) \dashv \vdash K_{\text{noexp}}^\uparrow(\mathbf{g} \wedge \mathbf{b})$, keeping all the outgoing edges of j . This replacement is possible because $\mathbf{g}$ is deducible and conditions **N4** and **N3** ensure that $\mathbf{b}$ is deducible whenever $\mathbf{a}^{-1} * \mathbf{b}$ is.

4) *Properties of Normal Dependency Graphs:* We prove two properties of normal dependency graphs that are crucial for our search algorithm. The first property states that every $K_e^\downarrow(t)$ -premise is deduced using a chain of deconstruction rules from a received message. We use here the *extended set of deconstruction rules* ND^{destr} that consists of the deconstruction and exponentiation rules from Figure 9. To define the second property, we partition the construction rules into the *implicit construction rules* $ND^{c\text{-impl}}$ consisting of the pair, inversion, and multiplication construction rules and the *explicit construction rules* $ND^{c\text{-expl}}$ consisting of the remaining construction rules and the **COERCE** rule. Since all messages that are products, pairs, and inverses must be constructed with implicit construction rules, we can show that if a $K_e^\uparrow(t)$ conclusion was deduced, then every message in $\text{inp}(t)$ must have been previously deduced.

Let $dg = (I, D)$ be a normal dependency graph for P . Its *deconstruction chain relation* $\rightarrow_{dg}$ is the smallest relation such that $c \rightarrow_{dg} p$ if c is a $K^\downarrow$ -conclusion in dg and (a) $c \rightsquigarrow p \in D$ or (b) there is a premise (j, u) such that $c \rightsquigarrow (j, u) \in D$ and $(j, 1) \rightarrow_{dg} p$. Our search algorithm exploits the following lemma to reason about the possible origins of $K^\downarrow$ -premises.

Lemma 2 (Deconstruction Chain). *For every premise p with*

fact $K_e^\downarrow(t)$ of dg , there is a node i in dg such that $I_i \in \text{ginsts}(\text{IRECV})$ and $(i, 1) \rightarrow_{dg} p$.

The *implicit construction dependency relation* $\rightarrow_{dg}$ of dg is the smallest relation such that $c \rightarrow_{dg} p$ if there is a premise (j, u) with $I_j \in \text{ginsts}(ND^{c\text{-impl}})$ such that (a) $c \rightsquigarrow (j, u) \in D$ and $(j, 1) \rightsquigarrow p \in D$ or (b) $c \rightarrow_{dg} (j, u)$ and $(j, 1) \rightsquigarrow p \in D$. Our algorithm uses the following lemma to keep the construction of pairs, inverses, and products implicit in the search.

Lemma 3 (Implicit Construction). *For every premise p in dg with fact $K_e^\downarrow(t)$ and every message $m \in_{AC} \text{inp}(t)$ with $m \neq_{AC} t$, there is a conclusion $(i, 1)$ in dg with fact $K_{e'}^\uparrow(m')$ such that $I_i \in \text{ginsts}(ND^{c\text{-expl}})$, $m' =_{AC} m$, and $(i, 1) \rightarrow_{dg} p$.*

Example 6. For the normal message deduction subgraph dg in Figure 8b, $(5, 1) \rightarrow_{dg} (9, 1)$, $(8, 1) \rightarrow_{dg} (9, 1)$, and $(6, 1) \rightarrow_{dg} (7, 1)$, but not $(6, 1) \rightarrow_{dg} (9, 1)$ because $(8, 2)$ is not a $K^\downarrow$ -premise. We have $(9, 1) \rightarrow_{dg} (11, 1)$, but not $(10, 1) \rightarrow_{dg} (11, 1)$ because $\rightarrow_{dg}$ requires at least one *inner* implicit construction node.

VI. AUTOMATED PROTOCOL ANALYSIS

In this section, we give an algorithm for determining whether $P \models_{EDH} \varphi$ for a $*$ -restricted protocol P and a guarded trace property φ . Guarded trace properties are an expressive subset of trace formulas. Our algorithm uses constraint solving to perform a complete search for counterexamples to $P \models_{EDH} \varphi$, i.e., it attempts a proof by contradiction. This problem is undecidable and our algorithm does

Trace formula reduction rules:

$\mathbf{S}_{\approx} :$	$\Gamma \rightsquigarrow_P \parallel_{\sigma \in \text{unify}_{AC}(t_1, t_2)} (\Gamma \sigma)$	if $(t_1 \approx t_2) \in \Gamma$ and $t_1 \neq_{AC} t_2$
$\mathbf{S}_{\doteq} :$	$\Gamma \rightsquigarrow_P \Gamma\{i/j\}$	if $(i \doteq j) \in \Gamma$ and $i \neq j$
$\mathbf{S}_{@} :$	$\Gamma \rightsquigarrow_P \parallel_{ri \in [P]^{DH} \cup \{\text{ISEND}\}} \parallel_{f' \in \text{acts}(ri)} (i : ri, f \approx f', \Gamma)$	if $(f @ i) \in \Gamma$ and $(f @ i) \notin_{AC} \text{as}(\Gamma)$
$\mathbf{S}_{\perp} :$	$\Gamma \rightsquigarrow_P \perp$	if $\perp \in \Gamma$
$\mathbf{S}_{\neg, \approx} :$	$\Gamma \rightsquigarrow_P \perp$	if $\neg(t \approx t) \in_{AC} \Gamma$
$\mathbf{S}_{\neg, \doteq} :$	$\Gamma \rightsquigarrow_P \perp$	if $\neg(i \doteq i) \in \Gamma$
$\mathbf{S}_{\neg, @} :$	$\Gamma \rightsquigarrow_P \perp$	if $\neg(f @ i) \in \Gamma$ and $(f @ i) \in \text{as}(\Gamma)$
$\mathbf{S}_{\neg, <} :$	$\Gamma \rightsquigarrow_P (i < j, \Gamma) \parallel (\Gamma\{i/j\})$	if $\neg(j < i) \in \Gamma$ and neither $i <_{\Gamma} j$ nor $i = j$
$\mathbf{S}_{\vee} :$	$\Gamma \rightsquigarrow_P (\phi_1, \Gamma) \parallel (\phi_2, \Gamma)$	if $(\phi_1 \vee \phi_2) \in_{AC} \Gamma$ and $\{\phi_1, \phi_2\} \cap_{AC} \Gamma = \emptyset$
$\mathbf{S}_{\wedge} :$	$\Gamma \rightsquigarrow_P (\phi_1, \phi_2, \Gamma)$	if $(\phi_1 \wedge \phi_2) \in_{AC} \Gamma$ and not $\{\phi_1, \phi_2\} \subseteq_{AC} \Gamma$
$\mathbf{S}_{\exists} :$	$\Gamma \rightsquigarrow_P (\phi\{y/x\}, \Gamma)$	if $(\exists x:s. \phi) \in \Gamma$, $\phi\{w/x\} \notin_{AC} \Gamma$ for every term w of sort s , and $y:s$ fresh
$\mathbf{S}_{\forall} :$	$\Gamma \rightsquigarrow_P (\psi\sigma, \Gamma)$	if $(\forall \vec{x}. \neg(f @ i) \vee \psi) \in \Gamma$, $\text{dom}(\sigma) = \text{set}(\vec{x})$, $(f @ i)\sigma \in_{AC} \text{as}(\Gamma)$, and $\psi\sigma \notin_{AC} \Gamma$

Graph constraint reduction rules:

$\mathbf{U}_{lbl} :$	$\Gamma \rightsquigarrow_P (ri \approx ri', \Gamma)$	if $\{i : ri, i : ri'\} \subseteq \Gamma$ and $ri \neq_{AC} ri'$
$\mathbf{DG1}_1 :$	$\Gamma \rightsquigarrow_P \perp$	if $i <_{\Gamma} i$
$\mathbf{DG1}_2 :$	$\Gamma \rightsquigarrow_P (f \approx f', \Gamma)$	if $c \rightsquigarrow p \in \Gamma$, $(c, f) \in \text{cs}(\Gamma)$, $(p, f') \in \text{ps}(\Gamma)$, and $f \neq_{AC} f'$
$\mathbf{DG2}_1 :$	$\Gamma \rightsquigarrow_P (\text{if } u = v \text{ then } \Gamma\{i/j\} \text{ else } \perp)$	if $\{(i, v) \rightsquigarrow p, (j, u) \rightsquigarrow p\} \subseteq \Gamma$ and $i \neq j$
$\mathbf{DG2}_{2,P} :$	$\Gamma \rightsquigarrow_P \parallel_{ri \in [P]^{DH} \cup \{\text{ISEND}, \text{FRESH}\}} \parallel_{u \in \text{idx}(\text{concs}(ri))} (i : ri, (i, u) \rightsquigarrow p, \Gamma)$ if p is an open f -premise in Γ , f is not a $\mathbf{K}^{\uparrow}$ - or $\mathbf{K}^{\downarrow}$ -fact, and i fresh	
$\mathbf{DG3} :$	$\Gamma \rightsquigarrow_P (\text{if } u = v \text{ then } \Gamma\{i/j\} \text{ else } \perp)$	if $\{c \rightsquigarrow (i, v), c \rightsquigarrow (j, u)\} \subseteq \Gamma$, c linear in Γ , and $i \neq j$,
$\mathbf{DG4} :$	$\Gamma \rightsquigarrow_P \Gamma\{i/j\}$	if $\{i : \neg[] \rightarrow \text{Fr}(m), j : \neg[] \rightarrow \text{Fr}(m)\} \subseteq_{AC} \Gamma$ and $i \neq j$
$\mathbf{N1} :$	$\Gamma \rightsquigarrow_P \perp$	if $(i : ri) \in \Gamma$ and ri not $\downarrow_{DH}$ -normal
$\mathbf{N5,6} :$	$\Gamma \rightsquigarrow_P \Gamma\{i/j\}$	if $\{((i, 1), \mathbf{K}_e^d(t)), ((j, 1), \mathbf{K}_{e'}^{d'}(t))\} \subseteq_{AC} \text{cs}(\Gamma)$, $i \neq j$, and $d = d'$ or $\{i, j\} \cap \{k \mid \exists ri \in \text{insts}(\{\text{PAIR} \uparrow, \text{INV} \uparrow, \text{COERCE}\}). (k : ri) \in \Gamma\} = \emptyset$
$\mathbf{N6} :$	$\Gamma \rightsquigarrow_P (i < j, \Gamma)$	if $((j, v), \mathbf{K}_e^{\uparrow}(t)) \in \text{ps}(\Gamma)$, $m \in_{AC} \text{inp}(t)$, $((i, u), \mathbf{K}_e^{\downarrow}(m)) \in \text{cs}(\Gamma)$, and not $i <_{\Gamma} j$
$\mathbf{N7} :$	$\Gamma \rightsquigarrow_P \perp$	if $(i : \mathbf{K}_{\text{exp}}^{\downarrow}(s_1), \mathbf{K}_e^{\uparrow}(t_1) \neg[] \rightarrow \mathbf{K}_{\text{noexp}}^{\downarrow}(s_2 \wedge t_2)) \in \Gamma$, s_2 is of sort <i>pub</i> , and $\text{inp}(t_2) \subseteq \text{inp}(t_1)$

Message deduction constraint reduction rules:

$\mathbf{DG2}_{2,\uparrow i} :$	$\Gamma \rightsquigarrow_P \parallel_{(l \neg[] \rightarrow \mathbf{K}_e^{\uparrow}(t)) \in \text{ND}^{c\text{-expl}}} (i : (l \neg[] \rightarrow \mathbf{K}_e^{\uparrow}(t)), t \approx m, (i, 1) \rightsquigarrow p, \Gamma)$ if p is an open implicit m -construction in Γ , m non-trivial, and i fresh	
$\mathbf{DG2}_{2,\uparrow e} :$	$\Gamma \rightsquigarrow_P \parallel_{ri \in \text{ND}^{c\text{-expl}}} (i : ri, (i, 1) \rightsquigarrow p, \Gamma)$ if p is an open $\mathbf{K}_e^{\uparrow}(m)$ -premise in Γ , $\{m\} = \text{inp}(m)$, m non-trivial, and i fresh	
$\mathbf{DG2}_{2,\downarrow} :$	$\Gamma \rightsquigarrow_P (i : \text{Out}(y) \neg[] \rightarrow \mathbf{K}_{\text{exp}}^{\downarrow}(y), (i, 1) \rightsquigarrow p, \Gamma)$	if p is an open $\mathbf{K}_e^{\downarrow}(m)$ -premise in Γ and y, i fresh
$\mathbf{DG2}_{\neg \rightarrow} :$	$(c \neg \rightarrow p, \Gamma) \rightsquigarrow_P (c \rightsquigarrow p, \Gamma) \parallel \parallel_{ri \in \text{ND}^{\text{destr}}} (i : ri, c \rightsquigarrow (i, 1), (i, 1) \neg \rightarrow p, \Gamma)$ if $(c, \mathbf{K}_e^{\downarrow}(m)) \in \text{cs}(\Gamma)$, $m \notin \mathcal{V}_{\text{msg}}$, and i fresh	

We assume that the multiset rewriting rules in $[P]^{DH}$, $\text{ND}^{c\text{-expl}}$, and ND^{destr} are renamed apart from Γ . We write $\Gamma\{a/b\}$ for the substitution of all occurrences of b with a in Γ . We write $\Gamma \rightsquigarrow_P \Gamma_1 \parallel \dots \parallel \Gamma_n$ for $\Gamma \rightsquigarrow_P \{\Gamma_1, \dots, \Gamma_n\}$, which denotes an n -fold case distinction. We overload notation and write $\perp$ for the empty set of constraint systems.

Figure 10. Rules defining the constraint-reduction relation $\rightsquigarrow_P$, explained in Sections VI-C and VI-D.

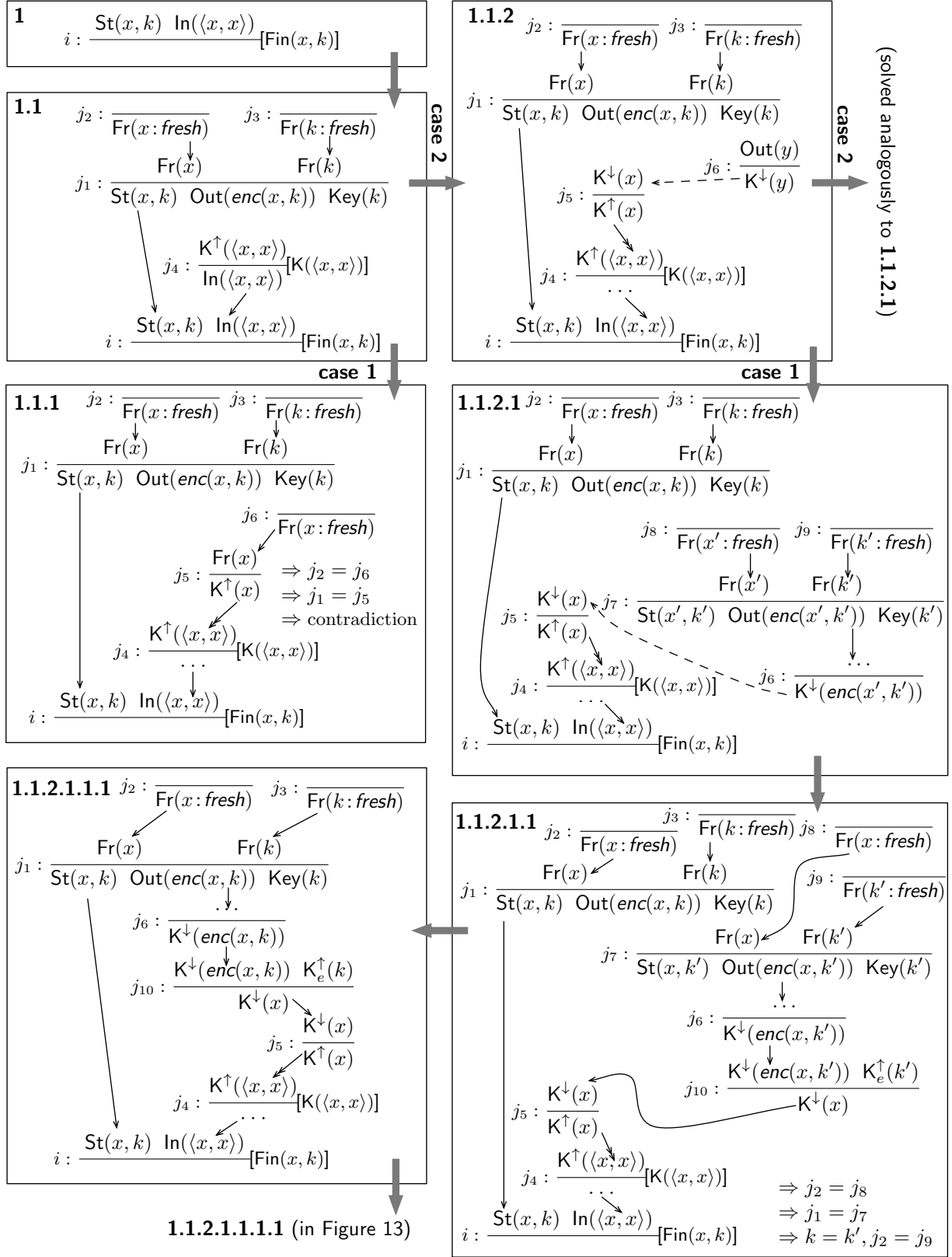


Figure 11. Constraint systems constructed by our algorithm, as explained in Example 7, when verifying $\forall x k i. \text{Fin}(x, k) @ i \Rightarrow \exists j. \text{Rev}(k) @ j$ for the protocol from Example 1. The large gray arrows denote constraint-reduction steps and “...” at either end of an edge refers to the fact at the other end. In every constraint system, variables with the same name are of the same sort and variables that have no sort annotation are of sort *msg*.

not always terminate. Nevertheless, it often finds a counter-example (an attack) or succeeds in unbounded verification.

In the following, we define guarded trace properties and constraints. Afterwards we give our constraint-solving algorithm and several examples.

A. Guarded Trace Properties

In the remainder of this section, let f range over facts and i, j over temporal variables. A trace formula φ is in *negation normal form* if it is built such that negation is only applied to trace atoms and all other logical connectives are $\wedge, \vee, \forall$, or $\exists$. Such a trace formula φ is a *guarded trace formula* if all its quantifiers are of the form $\exists \vec{x}. (f @ i) \wedge \psi$ or $\forall \vec{x}. \neg (f @ i) \vee \psi$ for an action $f @ i$, a guarded trace formula ψ , and $x \in \text{vars}(f @ i) \cap (\mathcal{V}_{msg} \cup \mathcal{V}_{temp})$ for every $x \in \vec{x}$. A guarded trace formula φ is a *guarded trace property* if it is closed and $t \in \mathcal{V} \cup PN$ holds for all terms t occurring in φ .

Note that we restrict both universal and existential quantification and, as a result, the set of guarded trace properties is closed under negation. This, together with the support for quantifier alternations and the explicit comparison of timepoints, makes guarded trace properties well-suited for specifying advanced security properties. In our case studies, it was possible to automatically convert the specified security properties, including the eCK model from Figure 4, to guarded trace properties. The conversion first rewrites the given formula to negation normal form and pushes quantifiers inward. Then, it replaces each body φ of a universal quantifier that is not a disjunction with $\varphi \vee \perp$. The rewriting for existential quantifiers is analogous.

All terms in a guarded trace property must be either variables or public names. This is not a limitation in practice since the terms required to express a security property can be added to the actions of a protocol's rewriting rules. Together with the requirement of guarding all quantified variables, this ensures that guarded trace properties are invariant under $\downarrow_{DH}$ -normalization of traces. Combined with Lemma 1, this allows us to switch from verification in a multiset rewriting semantics modulo E_{DH} to verification in a dependency graph semantics modulo AC .

Theorem 1. *For every $*$ -restricted protocol P and every guarded trace property φ ,*

$$P \models_{E_{DH}} \varphi \quad \text{iff} \quad \{ \text{trace}(dg) \mid dg \in \text{ndgraphs}(P) \} \models_{AC} \varphi.$$

B. Syntax and Semantics of Constraints

In the remainder of this section, let ri range over multiset rewriting rule instances, u and v over natural numbers, and φ over guarded trace formulas. A *graph constraint* is either a *node* $i : ri$, an *edge* $(i, u) \rightarrow (j, v)$, a *deconstruction chain* $(i, u) \rightarrow (j, v)$, or an *implicit construction* $(i, u) \rightarrow (j, v)$. A *constraint* is a graph constraint or a guarded trace formula.

A *structure* is a tuple (dg, θ) of a dependency graph $dg = (I, D)$ and a valuation θ . We denote the application of the

homomorphic extension of θ to a rule instance ri by $ri \cdot \theta$. We define when the structure (dg, θ) *satisfies* a constraint γ , written $(dg, \theta) \models \gamma$, as follows.

$$\begin{aligned} (dg, \theta) \models i : ri & \quad \text{iff } \theta(i) \in \text{idx}(I) \text{ and } ri \cdot \theta =_{AC} I_{\theta(i)} \\ (dg, \theta) \models (i, u) \rightarrow (j, v) & \quad \text{iff } (\theta(i), u) \rightarrow (\theta(j), v) \in D \\ (dg, \theta) \models (i, u) \rightarrow (j, v) & \quad \text{iff } (\theta(i), u) \rightarrow_{dg} (\theta(j), v) \\ (dg, \theta) \models (i, u) \rightarrow (j, v) & \quad \text{iff } (\theta(i), u) \rightarrow_{dg} (\theta(j), v) \\ (dg, \theta) \models \varphi & \quad \text{iff } (\text{trace}(dg), \theta) \models_{AC} \varphi \end{aligned}$$

A *constraint system* Γ is a finite set of constraints. The structure (dg, θ) *satisfies* Γ , written $(dg, \theta) \models \Gamma$, if (dg, θ) satisfies each constraint in Γ . We say that (dg, θ) is a *P-model* of Γ , if dg is a normal dependency graph for P and $(dg, \theta) \models \Gamma$. A *P-solution* of Γ is a normal dependency graph dg for P such that there is a valuation θ with $(dg, \theta) \models \Gamma$. Note that the free variables of a constraint system are therefore existentially quantified.

C. Constraint-Solving Algorithm

Let P be a $*$ -restricted protocol and φ a guarded trace property. Exploiting Theorem 1, our algorithm searches for a counter-example to $P \models_{E_{DH}} \varphi$ by trying to construct a *P-solution* to the constraint system $\{\hat{\varphi}\}$, where $\hat{\varphi}$ is $\neg \varphi$ rewritten into negation normal form. Our algorithm is based on the constraint-reduction relation $\sim_P$ between constraint systems and sets of constraint systems. We use sets of constraint systems to represent case distinctions.

Intuitively, $\sim_P$ refines constraint systems and our algorithm works by refining the initial constraint system $\{\hat{\varphi}\}$ until it either encounters a solved system or all systems contain (trivially) contradictory constraints. In the following, we first define $\sim_P$ and then state our algorithm. Afterwards, we give examples that explain and illustrate both the constraint-reduction rules defining $\sim_P$ and our algorithm.

The rules defining the *constraint-reduction relation* $\sim_P$ are given in Figure 10. There are two types of constraint-reduction rules: (1) *simplification rules* that remove contradictory constraint systems or refine constraint systems by simplifying constraints and (2) *case distinction rules* that refine constraint systems by adding further constraints. The design choices underlying our rules are motivated by their use in our algorithm. It requires them to be sound and complete and we must be able to extract a *P-solution* from every solved constraint system, i.e., every system that is irreducible with respect to $\sim_P$. The rule names refer to the form of guarded trace formulas that they solve or the property of normal dependency graphs that they ensure. There are no rules for ensuring the properties N2–4, as they are maintained as invariants by our algorithm.

The formal definition of our constraint-reduction rules in Figure 10 relies on the following additional conventions and definitions. We extend the equality $\approx$ over terms to facts and rule instances by interpreting the constructors for facts and

rule instances as free function symbols. We write $\text{PAIR}\uparrow$ for the construction rule for pairs and $\text{INV}\uparrow$ for the construction rule for inverses. Moreover, for a constraint system Γ , its *actions* $as(\Gamma)$, *premises* $ps(\Gamma)$, and *conclusions* $cs(\Gamma)$ are defined as follows.

$$\begin{aligned} as(\Gamma) &= \{f@i \mid \exists r a. (i : l \dashv [a] \mapsto r) \in \Gamma \wedge f \in a\} \\ ps(\Gamma) &= \{((i, u), l_u) \mid \exists r a. (i : l \dashv [a] \mapsto r) \in \Gamma \wedge u \in idx(l)\} \\ cs(\Gamma) &= \{((i, v), r_v) \mid \exists l a. (i : l \dashv [a] \mapsto r) \in \Gamma \wedge v \in idx(r)\} \end{aligned}$$

A conclusion c is *linear* in Γ if there is a linear fact f such that $(c, f) \in cs(\Gamma)$. We say that p is an *open f -premise* in Γ if $(p, f) \in ps(\Gamma)$ and p has no incoming edges or deconstruction chains in Γ . We say that p is an *open implicit m -construction* in Γ if there is a premise $(p, K_e^\dagger(t))$ of Γ with $m \in \text{inp}(t) \setminus \{t\}$ and there is no implicit construction $c \twoheadrightarrow p$ in Γ that starts from a $K_e^\dagger(m)$ -conclusion. A term m is *trivial* if $m \in \mathcal{V}_{msg} \cup \mathcal{V}_{pub} \cup PN \cup \{1\}$, where the term 1 denotes the unit in the group of exponents. The *temporal order* of Γ is

$$\begin{aligned} (\leq_\Gamma) &= \{(i, j) \mid (i \leq j) \in \Gamma \vee \exists u v. ((i, u) \twoheadrightarrow (j, v)) \in \Gamma \\ &\quad \vee ((i, u) \twoheadrightarrow (j, v)) \in \Gamma \\ &\quad \vee ((i, u) \rightarrow (j, v)) \in \Gamma\}^+ . \end{aligned}$$

We call $K^\dagger$ - and $K^\downarrow$ -premises *message deduction constraints*.

We give our constraint-solving algorithm in Figure 12. It uses a set of constraint systems as its state Ω . It starts with the state $\{\{\hat{\varphi}\}\}$. Afterwards, in lines 4–6, it repeatedly applies constraint-reduction steps as long as the state is non-empty and does not contain a solved constraint system. To formalize the loop condition, we use $\text{solved}(\Omega)$ to denote the set of solved constraint systems in Ω . For automated protocol analysis, we use a heuristic (explained in [18]) to make the choice in line 5. Upon termination of the while-loop, the algorithm has either found a solved constraint system (an attack) or it proved that $\{\{\hat{\varphi}\}\}$ has no P -solution and therefore $P \models_{E_{DH}} \varphi$ holds. The following two theorems justify the correctness of our algorithm.

Theorem 2. *The constraint-reduction relation $\sim_P$ is sound and complete; i.e., for every $\Gamma \sim_P \{\Gamma_1, \dots, \Gamma_n\}$, the set of P -solutions of Γ is equal to the union of the sets of P -solutions of all Γ_i , with $1 \leq i \leq n$.*

Theorem 3. *We can construct a P -solution from every solved system in the state Ω of our constraint-solving algorithm.*

The correctness of Theorem 3 relies on the properties of solved constraint systems as well as invariants maintained by our constraint-solving algorithm, as explained in [18].

D. Extended Examples

We now give two examples that illustrate our constraint-reduction rules. Our emphasis is on the rules and their application, rather than the heuristics used in our algorithm.

```

1: function SOLVE( $P \models_{E_{DH}} \varphi$ )
2:    $\hat{\varphi} \leftarrow \neg\varphi$  rewritten into negation normal form
3:    $\Omega \leftarrow \{\{\hat{\varphi}\}\}$ 
4:   while  $\Omega \neq \emptyset$  and  $\text{solved}(\Omega) = \emptyset$  do
5:     choose  $\Gamma \sim_P \{\Gamma_1, \dots, \Gamma_k\}$  such that  $\Gamma \in \Omega$ 
6:      $\Omega \leftarrow (\Omega \setminus \{\Gamma\}) \cup \{\Gamma_1, \dots, \Gamma_k\}$ 
7:   if  $\text{solved}(\Omega) \neq \emptyset$ 
8:     then return “attack(s) found: ”,  $\text{solved}(\Omega)$ 
9:   else return “verification successful”

```

Figure 12. Pseudocode of our constraint solving algorithm.

Example 7. Consider the protocol P from Example 1 and the formula $\forall x k i. \text{Fin}(x, k)@i \Rightarrow \exists j. \text{Rev}(k)@j$, which can be rewritten to the guarded trace property $\varphi = \forall x k i. \neg(\text{Fin}(x, k)@i) \vee (\exists j. \text{Rev}(k)@j \wedge \neg(\perp))$. We explain how to use our constraint-reduction rules to show that $P \models_{E_{DH}} \varphi$, i.e., to show that $\{\exists x k i. \psi\}$ has no P -solutions, for $\psi = \text{Fin}(x, k)@i \wedge (\forall j. \neg(\text{Rev}(k)@j) \vee \perp)$.

By repeated application of the rules $\mathbf{S}_\exists$ and $\mathbf{S}_\wedge$, we replace the existentially quantified variables x , k , and i with fresh free variables and split the conjunction in ψ . The resulting constraint system is

$$\begin{aligned} &\{\exists x k i. \psi, \exists k i. \psi, \exists i. \psi, \psi, \\ &\quad \text{Fin}(x, k)@i, \forall j. \neg(\text{Rev}(k)@j) \vee \perp\} . \end{aligned}$$

Its P -solutions are the normal dependency graphs for P that contain a node with a $\text{Fin}(x, k)$ action, but no node with a $\text{Rev}(k)$ action. Note that the formulas $\exists x k i. \psi$, $\exists k i. \psi$, $\exists i. \psi$, and ψ are solved in the sense that no reduction rule applies to them anymore.

The only rule applicable to the constraint system in the above state is $\mathbf{S}_@$, which enumerates all rewriting rules that could give rise to an action. We therefore apply $\mathbf{S}_@$ to the $\text{Fin}(x, k)$ action of node i and solve the introduced equalities using $\mathbf{S}_\sim$. There is only one resulting constraint system, depicted in Figure 11 as System 1. Here we interpret a constraint system as a partial, symbolic dependency graph annotated with restrictions on its trace, stated as guarded trace formulas. We do not depict the trace restriction $\forall j. \neg(\text{Rev}(k)@j) \vee \perp$, as it is included in all constraint systems. We also omit the exp tags of K -facts because the protocol does not use exponentiation.

The large gray arrows in Figure 11 denote the constraint-reduction steps performed. Multiple successors result from case distinctions. Constraint systems with no successors are contradictory for the reason given in the figure. In some cases, we contract multiple reduction steps for ease of presentation.

System 1 has multiple open non- K -premises. We solve these by repeatedly applying $\mathbf{DG2}_{2,P}$ until no more open non- K -premises remain. Intuitively, rule $\mathbf{DG2}_{2,P}$ solves open non- K -premises by enumerating all possible rewriting rules that have a unifying conclusion. The unification is entailed by the constraint $(i, u) \twoheadrightarrow p$ and the rules $\mathbf{DG1}_2$ and $\mathbf{S}_\sim$, which ensure the equality of facts connected by an edge. After

1.1.2.1.1.1 (in Figure 11)

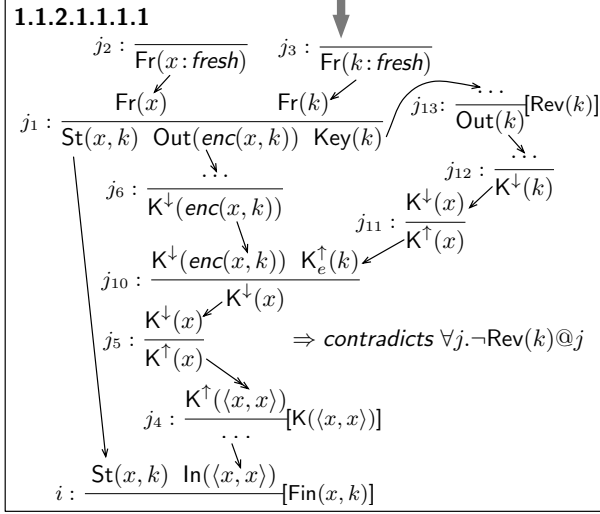


Figure 13. Example constraint system 1.1.2.1.1.1.

solving all non-K-premises, the only resulting constraint system is System 1.1.

There is only one remaining open premise in System 1.1: $K^{\uparrow}(\langle x, x \rangle)$. It is solved using rule $\mathbf{DG2}_{2,\uparrow i}$, which exploits Lemma 3 to search directly for the possible origin of the required input component $x: \text{fresh}$. This rule improves the efficiency of our algorithm as it allows reasoning modulo pairing, inversion, and multiplication. Note that the rules $\mathbf{DG2}_{2,\uparrow e}$ and $\mathbf{DG2}_{2,\uparrow i}$ exclude solving $K^{\uparrow}(x)$ premises where x is a variable of sort *msg* or *pub*, as such facts can always be constructed by the adversary. These rules may however become applicable once x is instantiated with another term in a later step. There are two multiset rewriting rules in ND whose conclusions are possible origins of $x: \text{fresh}$.

The case for the rule $(\text{Fr}(x: \text{fresh}) \rightarrow K^{\uparrow}_{\text{exp}}(x: \text{fresh}))$ is shown in System 1.1.1. Note that FRESH instances must be unique and linear facts have at most one outgoing edge. This is ensured by the rules $\mathbf{DG4}$ and $\mathbf{DG3}$, which together with the rule $\mathbf{U}_{lbl}$ give rise to the chain of implications depicted in System 1.1.1. The contradiction follows because the rewriting rule instances of nodes j_1 and j_5 are not unifiable.

The case for the COERCE rule is shown in System 1.1.2. Note that solving $K^{\downarrow}$ -premises by enumerating the rules with unifying conclusions leads to non-termination, as the $K^{\downarrow}$ -premises of deconstruction rules are larger than their conclusions. This justifies the existence of rule $\mathbf{DG2}_{2,\downarrow}$, which solves the $K^{\downarrow}(x)$ -premise by exploiting Lemma 2 to introduce a deconstruction chain starting from an IRECV instance. Rule $\mathbf{DG2}_{2,P}$ then solves the open premise $\text{Out}(y)$ of j_6 , i.e., it enumerates all protocol rules that send messages. There are two such rules in protocol P .

System 1.1.2.1 shows the case for the first protocol rule, which is renamed apart from System 1.1.2 since node j_7

might be a different rewriting rule instance than node j_1 . The deconstruction chain from j_6 to j_5 is refined by rule $\mathbf{DG2}_{2,\downarrow}$. This rule embodies the case distinction that a deconstruction chain is either just an edge or an edge to an instance of a rule from ND^{destr} and another chain starting from the conclusion of this instance. We disallow refining chains that start from a message variable, as this would lead to non-termination. Constraint systems containing such a chain are often pruned using rule $\mathbf{N6}$, as explained in Example 8. After using rule $\mathbf{DG2}_{2,\downarrow}$ to refine the deconstruction chain in System 1.1.2.1 twice, we obtain System 1.1.2.1.1 where x and x' are identified and the deconstruction chain has been replaced with the edges to and from the decryption j_{10} . This is the only way to refine this chain starting from $K^{\downarrow}(\text{enc}(x', k'))$, as all other deconstruction rules lead to cases with edges between non-unifiable facts. Again the uniqueness of FRESH instances leads to a chain of implications and results in System 1.1.2.1.1.1.

The only remaining open premise in this system is the $K^{\uparrow}(k)$ -premise of the decryption j_{10} . The constraint-reduction steps required to solve this premise are similar to the ones used to solve the $K^{\uparrow}(x)$ -premise in System 1.1. System 1.1.2.1.1.1 in Figure 13 is the only one of the resulting constraint systems that is not trivially contradictory due to the uniqueness of FRESH instances. In fact, this system could be instantiated to a normal dependency graph, if it were not for the trace restriction $\forall j. \neg(\text{Rev}(k) @ j) \vee \perp$. Since System 1.1.2.1.1.1 contains node j_{13} with a $\text{Rev}(k)$ action, we can use $\mathbf{S}_{\neg, @}$ and $\mathbf{S}_{\perp}$ to derive a contradiction. We first derive $\perp$ by applying $\mathbf{S}_{\neg, @}$ to the trace restriction $\forall j. \neg(\text{Rev}(k) @ j) \vee \perp$, instantiating j with j_{13} . Then, we apply rule $\mathbf{S}_{\perp}$. In general, we can always saturate constraint systems under universally quantified guarded trace formulas. This works because all trace formulas in a constraint system are guarded and the number of trace formulas derivable from a constraint system using just $\mathbf{S}_{\neg, @}$ is finite.

System 1.1.2.2 is also contradictory, which we show in [18]. Thus, we terminate without finding an attack and, as our search is complete, we therefore have a proof that $P \models_{E_{DH}} \varphi$.

The above example provides intuition for all rules except $\mathbf{N1}$, $\mathbf{N5,6}$, $\mathbf{N6}$, and $\mathbf{N7}$, which enforce normal-form message deduction. We explain them in the following paragraphs.

Rule $\mathbf{N1}$ ensures that all rule instances in node constraints are in $\downarrow_{DH}$ -normal form, i.e., it allows us to prune constraint systems containing DH -reducible terms. Intuitively, this rule prevents inconsistent instantiations of variables occurring in the variants of a multiset rewriting rule. Consider for example the multiset rewriting rule $\text{In}(x) \rightarrow \text{Out}(\text{fst}(x))$. The corresponding DH, AC -variants are $\text{In}(x) \rightarrow \text{Out}(\text{fst}(x))$ and $\text{In}(\langle y, z \rangle) \rightarrow \text{Out}(y)$. The rule $\mathbf{N1}$ allows pruning constraint systems where a node is labeled with the first variant and x is instantiated with a pair, as such constraint systems contradict the implicit assumption of the first rule variant, i.e., $\text{fst}(x)$

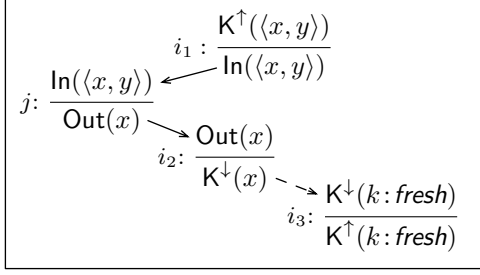


Figure 14. A contradictory constraint system where the adversary uses an instance of the protocol rule $\text{In}(\langle x, y \rangle) \rightarrow \text{Out}(x)$ to forward to himself the message x , which he deduced himself.

is not *DH*-reducible. Pruning constraint systems with *DH*-reducible terms is crucial when reasoning about the variants of DH exponentiations. Rule **N7** provides further support for reasoning about DH exponentiations, as it prunes constraint systems containing instances of deconstruction rules that can be replaced by instances of the construction rule for exponentiation.

Rule **N5,6** ensures that each $K^\uparrow$ - and $K^\downarrow$ -fact is derived, and therefore solved, at most once. Moreover, it ensures that every $K^\uparrow$ -premise deriving the same message as a $K^\downarrow$ -conclusion occurs after the $K^\downarrow$ -conclusion. This is required for Theorem 3 and allows pruning some constraint systems.

Rule **N6** prunes constraint systems where the adversary forwards a message via the protocol to himself. Such constraint systems occur when unfolding a deconstruction chain until it starts from a message variable whose content is received from the adversary. This is best seen in an example.

Example 8. Consider solving the premise $K^\uparrow(k:\text{fresh})$ in the context of a protocol that contains the multiset rewriting rule $\text{In}(\langle x, y \rangle) \rightarrow \text{Out}(x)$. One of the cases that we must consider is captured by the constraint system depicted in Figure 14. It states that the adversary might deduce k using a deconstruction chain starting from the message x sent by node j . Note that we suppress the exponentiation tags, as they are irrelevant for this example. This constraint system is contradictory because, in all its solutions, the adversary must deduce $K^\uparrow(x)$ before $K^\uparrow(\langle x, y \rangle)$, which implies that he cannot deduce $K^\downarrow(x)$ afterwards, as required by node i_3 . We show that this constraint system is contradictory using rule **DG1**₁ after we used rule **N6** to derive $i_3 < i_1$.

VII. CASE STUDIES

We implemented our constraint-solving algorithm in a tool, called the TAMARIN prover. It provides both a command-line interface and a graphical user interface, which allows to interactively inspect and construct attacks and proofs. We evaluated our algorithm on numerous protocols. Table I lists the results, run on a laptop with an Intel i7 Quad-Core processor. The tool and all models are available at [19].

We modeled the Signed Diffie-Hellman (SIG-DH) protocol, the STS protocol and two fixes [2], the UM [28], KEA+ [4],

Protocol	Security Model	Result	Time [s]
1. DH2 [31]	weakened eCK [31]	proof	6.7
2. KAS1 [30]	KI+KCI [31]	proof	0.3
3. KAS2 [30]	weakened eCK [31]	proof	2.9
4. KAS2 [30]	eCK	attack	0.4
5. KEA+ [4]	KI+KCI	proof	0.5
6. KEA+ [4]	KI+KCI+wPFS	attack	0.6
7. NAXOS [1]	eCK	proof	5.2
8. NAXOS [1]	eCK+PFS	attack on PFS	4.8
9. SIG-DH	PFS	proof	0.4
10. SIG-DH	eCK	attack	0.6
11. STS-MAC [2]	KI, reg-PK	UKS-attack	2.7
12. STS-MAC-fix1 [2]	KI, reg-PK (with PoP)	proof	8.6
13. STS-MAC-fix2 [2]	KI, reg-PK	proof	1.9
14. TS1-2004 [22]	KI	UKS-attack	0.2
15. TS1-2008 [29]	KI	proof	0.2
16. TS2-2004 [22]	KI+wPFS	attack on wPFS	0.4
17. TS2-2008 [29]	KI+wPFS	proof	0.7
18. TS3-2004/08 [22], [29]	KI+wPFS	non-termination	-
19. UM [28]	wPFS	proof	0.7
20. UM [28]	PFS	attack	0.4

Table I
RESULTS OF CASE STUDIES

and NAXOS [1] protocols, and the TS1, TS2, and TS3 protocols [22] and their updated versions [29]. We also modeled NIST's KAS1 and KAS2 protocols [30] and the related DH2 protocol by Chatterjee et. al. [31]. For each protocol, we formalized its intended and related security models and analyzed them using TAMARIN. For example, to verify Key Independence (KI) for STS, we model that the adversary can reveal certain session keys. Additionally, the adversary can register public keys for himself, even if those keys have been previously registered for another identity. In this example, we find the UKS attack reported in [2]. The first fix from [2] requires a Proof-of-Possession (PoP) of the private key for registering a public key. The second fix includes the identities of the participants in the signatures. We model and successfully verify both fixes. For NIST's KAS1 and KAS2 protocols [30], our analysis confirms both the security proof and the informal statements made in [31].

Our results indicate that, in general, TAMARIN is effective and efficient. For example, it requires 5.2 seconds to verify NAXOS in the eCK model (see Figure 3).

In general, there are two sources of non-termination of our algorithm. First, if the protocol contains loops (e.g., a rule like $A(x) \rightarrow A(h(x))$), then the reduction rule **DG2**_{2,P} can be applied infinitely often during backwards search. Reasoning about such protocols requires support for loop invariants, as used in program verification. Second, if the protocol can serve as a generic message deduction oracle, then our normal-form conditions may fail to eliminate sufficiently many redundant steps. This explains the non-termination for the TS3-2004/08 protocols. It remains future work to develop normal-form conditions that improve reasoning about such protocols.

VIII. RELATED WORK

Corin et. al. [32] and Armando et. al. [33] use linear temporal logics and constraint solving for security protocol

verification for a bounded number of sessions. Chevalier et. al. [7] and Shmatikov [8] prove that secrecy is decidable for a bounded number of sessions for DH theories similar to ours. Meadows et. al. [34] and Kapur et. al. [35] present unification algorithms for a DH theory similar to ours. In [35], Kapur et. al. show the undecidability of unification modulo a DH theory that also allows addition of exponents.

[12]–[14] support verification for an unbounded number of sessions, but do not consider inverses. Blanchet et. al. [12] extend ProVerif [36] to handle the property that $(x^y)^z \simeq (x^z)^y$. Goubault-Larrecq [13] accounts for this property using a Horn-theory approach and resolution modulo AC . Escobar et. al. [14] use Maude-NPA and equational unification to analyze secrecy properties of DH protocols. Since Maude-NPA supports user-specified equational theories, the verification problem with respect to our DH theory can be specified. It is however unclear if Maude-NPA can achieve unbounded verification for such a theory. In the free term algebra, Basin and Cremers [15] present models and tool support for compromising adversaries, based on Scyther [37].

Küsters and Truderung [6] give a transformation that, given a Horn theory modeling secrecy and simple authentication properties modulo a DH theory with inverses, produces a Horn theory in the free algebra, which they analyze using ProVerif. Their reduction is similar to our reduction from E_{DH} to AC , but works only for Horn clauses with ground exponents. As stated in [6], stronger security properties often violate this restriction. Since our approach allows for non-ground exponents, we can also find attacks where the adversary sends products, e.g., a protocol that receives a message x and leaks a secret if $g^{(a * b * x^{-1})} = g$.

Lynch and Meadows [9] and Mödersheim [10] give reductions for DH reasoning without inverses to reasoning modulo a simpler equational theory for a restricted class of protocols. Both require that all exponents used by a protocol remain secret forever. This excludes modeling ephemeral key reveals and thus verifying recent AKE protocols. Ngo et. al. [11] propose a method for the automated construction of computational proofs for a restricted class of DH-based protocols.

IX. CONCLUSION

We gave a novel constraint-solving algorithm and demonstrated its effectiveness in non-trivial case studies. Our algorithm exploits a special representation of message deduction that satisfies a “deconstruction chain” property, inspired by the theory underlying Athena [38] and Scyther [37], [39]. This property is key for achieving unbounded verification. It enables a backwards exploration of the interleavings of protocol steps guided by the solving of message deduction constraints. We constructed this representation by imposing normal-form conditions on the use of the (finite) variants of the message deduction rules. This construction and our

algorithm should also work for other theories with the finite variant property, e.g., theories for XOR and blind-signatures.

Although we were motivated by the verification of AKE protocols, neither our protocol model nor our constraint-solving algorithm are tailored to them. We expect both our model and algorithm to be applicable to a wide range of security protocol verification problems. Due to our multiset-rewriting model, our approach is especially promising for verifying protocols that exploit internal state, which are often hard to analyze using the Horn-theory approach. We plan to investigate such stateful protocols in future work together with support for loop invariants.

REFERENCES

- [1] B. LaMacchia, K. Lauter, and A. Mityagin, “Stronger security of authenticated key exchange,” in *Provable Security*, ser. LNCS, vol. 4784. Springer, 2007, pp. 1–16.
- [2] S. Blake-Wilson and A. Menezes, “Unknown Key-Share Attacks on the Station-to-Station (STS) Protocol,” in *Proceedings of the 2nd International Workshop on Practice and Theory in Public Key Cryptography*. Springer, 1999, pp. 154–170.
- [3] C. Cremers, “Session-StateReveal is stronger than eCK’s EphemeralKeyReveal: Using automatic analysis to attack the NAXOS protocol,” *International Journal of Applied Cryptography (IJACT)*, vol. 2, pp. 83–99, 2010.
- [4] K. Lauter and A. Mityagin, “Security analysis of KEA authenticated key exchange protocol,” in *PKC 2006*, ser. LNCS, vol. 3958. Springer, 2006, pp. 378–394.
- [5] M. Just and S. Vaudenay, “Authenticated multi-party key agreement,” in *ASIACRYPT 1996*, ser. LNCS, vol. 1163. Springer, 1996, pp. 36–49.
- [6] R. Küsters and T. Truderung, “Using ProVerif to analyze protocols with Diffie-Hellman exponentiation,” in *Proceedings of the 22nd IEEE Computer Security Foundations Symposium*. IEEE Computer Society, 2009, pp. 157–171.
- [7] Y. Chevalier, R. Küsters, M. Rusinowitch, and M. Turuani, “Deciding the security of protocols with Diffie-Hellman exponentiation and products in exponents,” in *FSTTCS 2003*, ser. LNCS, vol. 2914. Springer, 2003, pp. 124–135.
- [8] V. Shmatikov, “Decidable analysis of cryptographic protocols with products and modular exponentiation,” in *Programming Languages and Systems*, ser. LNCS. Springer, 2004, vol. 2986, pp. 355–369.
- [9] C. Lynch and C. Meadows, “Sound approximations to Diffie-Hellman using rewrite rules,” in *Information and Communications Security*, ser. LNCS. Springer, 2004, vol. 3269, pp. 65–70.
- [10] S. Mödersheim, “Diffie-Hellman without difficulty,” in *Proceedings of the 8th International Workshop on Formal Aspects of Security & Trust (FAST)*, 2011.
- [11] L. Ngo, C. Boyd, and J. Nieto, “Automated proofs for Diffie-Hellman-based key exchanges,” in *Proceedings of the 23rd IEEE Computer Security Foundations Symposium*, 2011, pp. 51–65.

- [12] B. Blanchet, M. Abadi, and C. Fournet, "Automated verification of selected equivalences for security protocols," *Journal of Logic and Algebraic Programming*, vol. 75, no. 1, pp. 3–51, 2008.
- [13] J. Goubault-Larrecq, M. Roger, and K. Verma, "Abstraction and resolution modulo AC: How to verify Diffie-Hellman-like protocols automatically," *Journal of Logic and Algebraic Programming*, vol. 64, no. 2, pp. 219–251, 2005.
- [14] S. Escobar, C. Meadows, and J. Meseguer, "State space reduction in the Maude-NRL protocol analyzer," in *Computer Security - ESORICS 2008*, ser. LNCS. Springer, 2008, vol. 5283, pp. 548–562.
- [15] D. Basin and C. Cremers, "Modeling and analyzing security in the presence of compromising adversaries," in *Computer Security - ESORICS 2010*, ser. LNCS, vol. 6345. Springer, 2010, pp. 340–356.
- [16] —, "Degrees of security: Protocol guarantees in the face of compromising adversaries," in *19th EACSL Annual Conference on Computer Science Logic (CSL)*, ser. LNCS, vol. 6247. Springer-Verlag, 2010, pp. 1–18.
- [17] H. Andr  ka, I. N  meti, and J. van Benthem, "Modal languages and bounded fragments of predicate logic," *Journal of Philosophical Logic*, vol. 27, pp. 217–274, 1998.
- [18] B. Schmidt, S. Meier, C. Cremers, and D. Basin, "Automated Analysis of Diffie-Hellman Protocols and Advanced Security Properties (Extended Version)," April 2012, available <http://www.infsec.ethz.ch/research/software#TAMARIN>.
- [19] S. Meier and B. Schmidt, "The TAMARIN prover: source code and case studies," April 2012, available <http://hackage.haskell.org/package/tamarin-prover-0.4.0.0>.
- [20] H. Krawczyk, "HMQV: A high-performance secure Diffie-Hellman protocol," in *Advances in Cryptology—CRYPTO 2005*, ser. LNCS, vol. 3621. Springer, 2005, pp. 546–566.
- [21] C. Cremers, "Examining indistinguishability-based security models for key exchange protocols: the case of CK, CK-HMQV, and eCK," in *Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security*. ACM, 2011, pp. 80–91.
- [22] I. R. Jeong, J. Katz, and D. H. Lee, "One-round protocols for two-party authenticated key exchange," in *Applied Cryptography and Network Security*. Springer, 2004, pp. 220–232.
- [23] F. J. Thayer, J. C. Herzog, and J. D. Guttman, "Strand spaces: Proving security protocols correct," *Journal of Computer Security*, vol. 7, no. 1, 1999.
- [24] S. Escobar, R. Sasse, and J. Meseguer, "Folding variant narrowing and optimal variant termination," *Rewriting Logic and Its Applications*, pp. 52–68, 2010.
- [25] H. Comon-Lundh and S. Delaune, "The finite variant property: How to get rid of some algebraic properties," *Term Rewriting and Applications*, pp. 294–307, 2005.
- [26] L. Law, A. Menezes, M. Qu, J. Solinas, and S. Vanstone, "An efficient protocol for authenticated key agreement," *Designs, Codes and Cryptography*, vol. 28, pp. 119–134, 2003.
- [27] E. Clarke, S. Jha, and W. Marrero, "Verifying security protocols with Brutus," *ACM Trans. Softw. Eng. Methodol.*, vol. 9, no. 4, pp. 443–487, 2000.
- [28] S. Blake-Wilson and A. Menezes, "Authenticated Diffie-Hellman key agreement protocols," in *Selected Areas in Cryptography*, ser. LNCS. Springer, 1999, vol. 1556, pp. 630–630.
- [29] I. R. Jeong, J. Katz, and D. H. Lee, "One-round protocols for two-party authenticated key exchange (full)," 2008, http://www.cs.umd.edu/~jkatz/papers/1round_AKE.pdf.
- [30] *SP 800-56B, Special Publication 800-56B, Recommendation for Pair-Wise Key Establishment Schemes using Integer Factorization Cryptography*, National Institute of Standards and Technology, August 2009.
- [31] S. Chatterjee, A. Menezes, and B. Ustaoglu, "A generic variant of NIST's KAS2 key agreement protocol," in *Proceedings of the 16th Australasian conference on Information security and privacy (ACISP'11)*. Springer, 2011, pp. 353–370.
- [32] R. Corin, A. Saptawijaya, and S. Etalle, "A logic for constraint-based security protocol analysis," in *Proceedings of IEEE Symposium on Security and Privacy, S&P 2006*, 2006, pp. 155–168.
- [33] A. Armando, R. Carbone, and L. Compagna, "LTL model checking for security protocols," in *Proceedings of the 23rd IEEE Computer Security Foundations Symposium*, 2007, pp. 385 – 396.
- [34] C. Meadows and P. Narendran, "A unification algorithm for the Group Diffie-Hellman protocol," in *Proc. of WITS 2002*, 2002.
- [35] D. Kapur, P. Narendran, and L. Wang, "An E-unification algorithm for analyzing protocols that use modular exponentiation," in *Rewriting Techniques and Applications*. Springer, 2003, pp. 165–179.
- [36] B. Blanchet, "An efficient cryptographic protocol verifier based on Prolog rules," in *Proceedings of the 14th IEEE Computer Security Foundations Workshop*. IEEE, 2001, pp. 82–96.
- [37] C. Cremers, "The Scyther Tool: Verification, falsification, and analysis of security protocols," in *Computer Aided Verification (CAV)*, ser. LNCS, vol. 5123. Springer, 2008, pp. 414–418.
- [38] D. Song, S. Berezin, and A. Perrig, "Athena: A novel approach to efficient automatic security protocol analysis," *Journal of Computer Security*, vol. 9, pp. 47–74, 2001.
- [39] S. Meier, C. Cremers, and D. A. Basin, "Strong invariants for the efficient construction of machine-checked protocol security proofs," in *Proceedings of the 23rd IEEE Computer Security Foundations Symposium*. IEEE Computer Society, 2010, pp. 231–245.